

Chapter 3: Worked Answers

Elmasri and Navathe, Fundamentals of Database Systems, seventh edition. Source: the instructor's photographed Chapter 3, printed pp.89-135. Answer 3.20 also uses Chapters 1 and 2 of the same-edition textbook. These are original worked answers, not an official publisher solution manual. Question numbers refer to the photographed copy; question statements are not reproduced. All invented records are illustrative. Open design questions admit other answers when their assumptions and constraints are stated.

Reading the Diagrams

Diagrams use ER rectangles and diamonds unless explicitly marked UML. Each (min,max) beside an entity counts that entity's participation in that relationship. Repeated entity names in separate rows mean the same type. In these min-max diagrams the numbers determine participation: zero permits absence, and a positive minimum requires participation (3.7.4). A single connector in this notation does not override a positive minimum. Where double connectors are retained for identification, they reinforce the same requirement. Double rectangles indicate weak types; double diamonds indicate identifying relationships. Complete keys and partial keys are listed in accompanying tables. An omitted constraint is not permission to silently invent a requirement. The lab answers provide conceptual diagrams. The modeling-application requirement in 3.31-3.35 is not completed: ERwin and Rational Rose have not been run, and no native model files from those tools are supplied. These answers do not add assignments or examination topics.

3.1. Conceptual Models in Database Design

Source: 3.1, pp.90-92.

Step	Output	Example
Collect requirements	Data and operation requirements	Each enrollment connects a student and a section.
Conceptual design	DBMS-independent schema	STUDENT -- TAKES -- SECTION; Grade belongs to TAKES.
Logical design	Schema in the selected data model	Relations and foreign keys, handled later in Ch9
Physical design	Storage and access choices	Files and indexes

A high-level model expresses objects, properties, associations, and constraints in terms users can check. It helps discover missing facts before implementation. Operations such as retrieving a student's grades check whether the conceptual schema contains enough information. An ER model is not an index design or a GUI.

Conceptual design precedes implementation

Question	Decision level
What objects and constraints must be represented?	Conceptual schema
How will that model use relations?	Logical schema
How will records be stored and accessed?	Physical design

A conceptual ER model does not choose an index or storage format.

3.2. Meanings of NULL

Source: 3.3.1, p.96.

Situation	Example	What is known
Not applicable	Apartment number for a detached house	No applicable value exists.
Unknown, value exists	An adult's height has not been measured	A height exists; its value is missing.
Unknown whether applicable	Home phone not recorded	We do not know whether a home phone exists.

NULL is not automatically zero, an empty string, or an empty list. These cases describe conceptual meanings; a stored SQL NULL does not by itself retain which reason caused the absence.

3.3. Basic Terms

Source: 3.3-3.4, pp.93-108.

Term	Meaning	Small example
Entity	Distinguishable object represented by the model	Student S1
Attribute	Property of an entity or relationship	Student's name
Attribute value	A particular value of that property	Alex
Relationship instance	Association of particular participants	S1 takes section Q1
Composite attribute	Attribute with meaningful components	Name(First, Last)
Multivalued attribute	Multiple values for one entity	Two phone numbers for S1
Derived attribute	Value calculated from other information	EnrollmentCount from TAKES
Complex attribute	Nested composite/multivalued structure	Several addresses, each with several phones
Key attribute	Minimal identifying attribute, possibly composite	StudentId
Value set/domain	Allowed attribute values	Credits: integers 1 through 6 in an example

Composite and multivalued are independent properties: Name can be composite and single-valued; Phone can be simple and multivalued.

3.4. Entity Type and Entity Set

Source: 3.3.2, pp.97-99.

An entity type specifies the common attributes of a collection of entities. An entity set is the collection of entities of that type currently represented in the database. An entity is one distinguishable object in that collection.

Level	Example	Changes when S3 registers?
Entity	S1, one student	S1 need not change.
Entity type	STUDENT(StudentId, Name)	No schema change is required.
Entity set	Current objects S1 and S2	Yes: now S1, S2, S3.

The type supplies a schema (intension); the set supplies its current extension. A sample set cannot establish that an attribute will always be unique.

3.5. Attribute Versus Value Set

Source: review question 3.5, p.126; 3.3.2, pp.99-100.

An attribute is a property being represented. Its value set, also called its domain, specifies the values allowed for that property. An attribute value is one actual value, not the whole domain.

Attribute, value set, and actual value are different

What is being named?	Illustrative COURSE example
Attribute: a property	Credits
Value set: allowed values	{1, 2, 3, 4, 5, 6}
Actual value for C1	3
Invalid value under this domain	7

The domain is not limited to the values currently stored in the database.

For the illustrative COURSE.Credits attribute, choose the integer domain {1,2,3,4,5,6}. Course C1 can have Credits = 3. The property is Credits, its domain contains six allowed values, and this course's actual value is 3. If the current courses use only 3 and 4, the domain still includes 1, 2, 5, and 6. A value of 7 violates this declared domain; the example does not set university credit policy. Different attributes can share a domain without becoming the same property.

3.6. Relationship Type, Set, and Instance

Source: 3.4.1, pp.102-103.

A relationship type describes associations among specified participating entity types. A relationship instance is one such association of particular entities; the relationship set contains all its current instances.

Level	Example
Type	TAKES between STUDENT and SECTION
Set	{{(S1,Q1), (S1,Q2), (S2,Q1)}} at the current time
Instance	(S1,Q2)

There are three instances, not three relationship types. A Grade value can be attached to an instance. The relationship set is a subset of the Cartesian product of its participating entity sets, with roles fixing participant order.

3.7. Participation Roles

Source: 3.4.2, pp.104-105.

A role states how an entity participates. In SUPERVISION, an employee can participate as supervisor or supervisee. Roles are essential when the same type participates more than once; they can also improve clarity elsewhere.

One EMPLOYEE type, two roles

ER: counts beside each entity



Left: supervisor role. Right: supervisee role. The two rectangles denote the same type.

For (E1,E2), E1 supervises E2. Reversing the pair changes the fact. One EMPLOYEE rectangle does not make this a unary relationship.

3.8. Two Constraint Notations

Source: 3.4.3 and 3.7.4, pp.106-108,114-115; 3.9.2, pp.121-122.

Notation	Strength	Limitation
1:1 / 1:N / M:N plus single/double lines	Separates maximum cardinality and optional/required participation	Does not express an exact bound such as 2 through 4 using only 1/M/N and line style.
(min,max)	States bounds for each entity directly	Label placement differs; on higher-degree relationships it does not replace fixed-other-participant key constraints.

Example: each department employs at least four employees; each employee works for exactly one department. Min-max puts (4,N) beside DEPARTMENT and (1,1) beside EMPLOYEE. A 1:N diagram puts 1 near DEPARTMENT, N near EMPLOYEE, and total participation lines on both; it loses the minimum of four.

Department minimum four is explicit

ER: counts beside each entity



Counts belong to the entity beside each pair. See the accompanying keys and assumptions.

For a ternary relationship, "at most one supplier per project-part pair" and "at most five supply facts per supplier" constrain different things.

3.9. Moving a Relationship Attribute

Source: 3.4.4, p.108. This is conceptual placement, not a Ch9 SQL mapping.

Relationship	Permitted entity placement	Why
1:1 MANAGES, StartDate	Either participating type	Each participant occurs in at most one instance.
1:N WORKS_FOR, StartDate	Only EMPLOYEE, the N side	An employee identifies one employment; a department has many.
M:N WORKS_ON, Hours	Keep on WORKS_ON when hours depend on both participants	Neither participant alone identifies the value.

Why one department date loses information

EMPLOYEE placement: keeps both dates

Employee	StartDate
E1	2025-01-01
E2	2025-04-01

DEPARTMENT placement: cannot choose

Department	StartDate
D1	January 1 or April 1?

E1 and E2 both work for D1. One single-valued date on D1 cannot retain who started when.

Employee	Department	StartDate
E1	D1	2025-01-01
E2	D1	2025-04-01

One StartDate on D1 loses which employee started when. Values need not be equal merely because the same department participates. Optional relationships require the entity placement to accommodate absence. Storing history can invalidate a 1:1 assumption, so this answer concerns the stated current-state relationships.

3.10. Relationships Represented as Attributes

Source: 3.4.2, p.104.

For WORKS_FOR, Employee.Department is a reference to a DEPARTMENT object. Conversely, Department.Employees can be a set of references to EMPLOYEE objects. Let D be the current DEPARTMENT entity set and E the EMPLOYEE entity set. Employee.Department takes values in D; Department.Employees takes values in $P(E)$, the power set of E (the set of all subsets of E). For $E = \{E1, E2\}$, $P(E) = \{\text{empty set}, \{E1\}, \{E2\}, \{E1, E2\}\}$. Thus $\{E1, E2\}$ is one possible value of Department.Employees, not one employee. If both directions are stored, they must be inverses: $E1.Department = D1$ exactly when $E1$ is in $D1.Employees$. This viewpoint is used in functional data models, as identified in the textbook's footnote on p.104. A reference to an entity is not merely a duplicated department name.

3.11. Recursive Relationships

Source: 3.4.2, p.105.

A type participates in multiple roles in the same relationship. Examples: EMPLOYEE supervises EMPLOYEE; PERSON is parent of PERSON; COURSE requires COURSE. The parent-child example is binary. It needs parent and child roles, and its maximum/minimum counts depend on stated requirements. The ER lines alone do not prohibit self-parenthood or cycles; add those constraints explicitly. See the role diagram in Answer 3.7.

3.12. Weak Entity Types

Source: 3.5, p.109.

Use a weak type when its attributes alone cannot identify its objects, but its owner identity together with a local discriminator can.

Concept	Definition	Example
Owner entity type	Supplies entity identity used to identify the weak entity	BANK, identified by Code
Weak entity type	Has no identifying key of its own	BANK_BRANCH
Identifying relationship	Connects the weak entity to the owner used in its identity	BRANCHES
Partial key	Distinguishes weak entities belonging to the same owner	BranchNo, unique only within a bank
Complete identity	Owner identity together with the partial key in this example	(bank Code, BranchNo)

BANK plus local BranchNo identifies a branch

ER: counts beside each entity



Key: BANK.Code. Partial key: BANK_BRANCH.BranchNo. Owner minimum is illustrative here, not Figure 3.22.

(B1,7) and (B2,7) are different branches. Double rectangles/diamonds identify weak types/identifying relationships; the partial key gets a dashed underline. The weak type participates totally in identification. A driver's license with a globally unique LicenseNo is regular despite requiring a person. Owners can themselves be weak; some multiple-owner designs need no partial key.

3.13. Higher-Degree Identifying Relationships

Source: 3.9.1, pp.120-121, Figure 3.19.

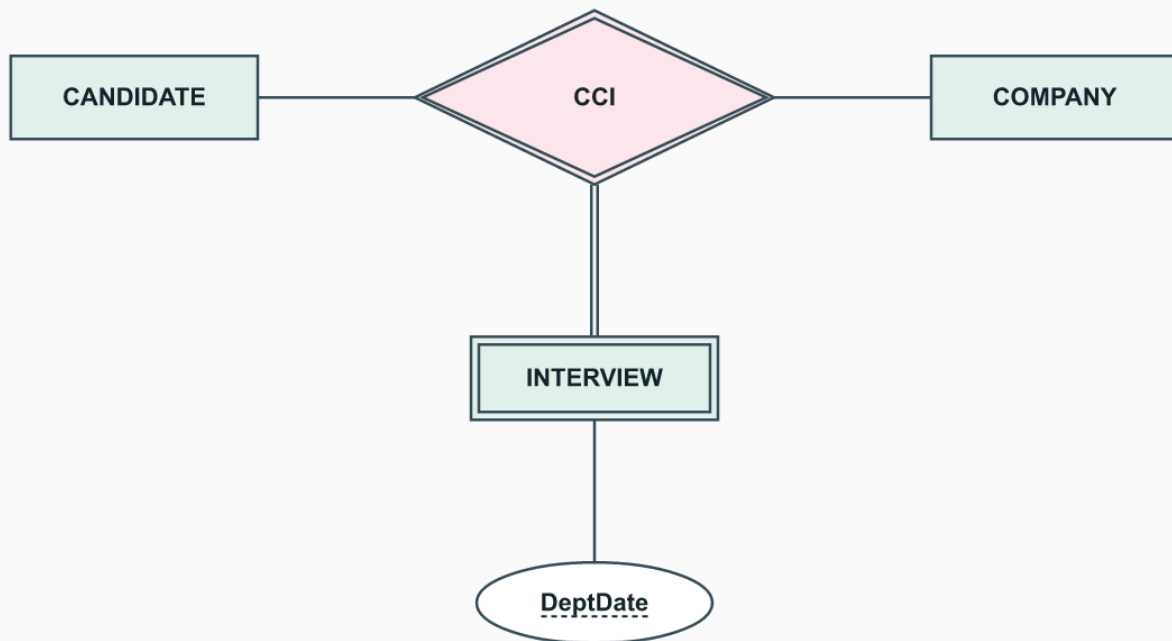
Yes. INTERVIEW can be identified by CANDIDATE, COMPANY, and local DeptDate(Department, Date). Its identifying relationship has three participants, including the weak type itself.

Candidate	Company	Department	Date
A1	C1	IT	2026-09-01
A1	C2	IT	2026-09-01
A1	C1	Finance	2026-09-01

All three interviews differ. The assumed local uniqueness rule must exclude two interviews for the same candidate/company/department/day, or include another discriminator such as start time. Figure 3.19's JOB_OFFER relationship explains why an interview may need its own identity rather than remaining only a property.

INTERVIEW has two identifying owners

ER: ternary identifying relationship



Candidate + Company + DeptDate identifies an interview. DeptDate is a partial key.

A second, invented example is VISIT owned jointly by PATIENT and CLINIC. Assume VisitNo is numbered separately for each patient-clinic pair. A ternary identifying relationship connects PATIENT, CLINIC, and weak VISIT; the complete identity is (PatientId,ClinicId,VisitNo). (P1,C1,1) and (P1,C2,1) then identify different visits. This numbering rule is a teaching assumption, not a medical rule.

3.14. ER Diagram Conventions

Source: 3.7.1, pp.111-113, Figure 3.14.

Mark	Meaning
Rectangle	Regular entity type
Double rectangle	Weak entity type
Diamond	Relationship type
Double diamond	Identifying relationship
Oval	Attribute
Underlined attribute	Key; separately underlined attributes are independent keys
Dashed-underlined attribute	Partial key
Double oval	Multivalued attribute
Dashed oval	Derived attribute
Parent oval with component ovals	Composite attribute
Single/double participation line	Partial/total participation
Role name beside a participation	Meaning of that participant, such as supervisor or supervisee
1, M, N	Maximum-ratio convention
(min,max) beside a type	Number of instances in which each entity participates

Lines connect named endpoints; crossing lines do not create unnamed entities or relationships. The symbols summarize a schema, not the current entity set.

3.15. Naming Conventions

Source: 3.7.2, p.112.

Construct	Recommended choice	Example
Entity type	Singular noun, uppercase	EMPLOYEE
Relationship	Descriptive verb phrase, uppercase	WORKS_FOR
Attribute	Meaningful name, initial capital	StartDate
Role	Lowercase role name	supervisor

Choose a reading direction that makes the relationship name intelligible. EMPLOYEE HAS_DEPENDENTS DEPENDENT and DEPENDENT DEPENDENTS_OF EMPLOYEE can express the same association. A noun in requirements is a candidate for analysis, not a mechanical rule that every noun becomes an entity type.

3.16. Additional UNIVERSITY Constraints

Source: exercise 3.16, pp.126-127; 3.10, pp.122-124.

Let Sid identify a student, SecId a section, Id an instructor, and Sem/Year the section's term. "X determines Y" means identical X values must have identical Y values under the chosen rule.

Part	Required rule	Correct placement
a	(Sid,SecId) determines Grade	One grade on each TAKES instance; absent until available
b	(Sid,Sem,Year,DaysTime) determines SecId	Cross-relationship scheduling constraint involving TAKES and SECTION
c	SecId determines Id	Already supplied by SECTION (1,1) in TEACHES

For c, (Sid,SecId) also determines Id, but Sid is unnecessary: a section already has only one instructor. Sem and Year are likewise unnecessary in a complete section identifier because SecId is global across terms.

Proposed facts	Result
S1,Q1,80 and S1,Q1,90 in the same grade state	Reject conflicting grades.
S1 takes Q1 and Q2 in Fall 2026 at the same DaysTime	Reject the scheduling conflict.
Q1 taught by I1 and I2	Reject, even before considering any student.

Sid is not an ordinary SECTION attribute. Thus (b) is not simply another underlined key oval on SECTION: it constrains an enrollment joined to its section's term and time. Exact time-label equality is insufficient for arbitrary overlapping intervals. Retakes in distinct sections are distinguishable by SecId. Neither (a) nor (b) can be enforced by a key made only from SECTION attributes: both involve the student participating in TAKES. For another constraint, under exact meeting-slot labels and no combined sections, (Bldg,RoomNo,Sem,Year,DaysTime) determines SecId. This prevents two sections from using one room in that slot; overlapping unequal intervals still need a separate check.

3.17. A Nested Job-History Attribute

Source: exercise 3.17, p.127; Figure 3.5, p.97. Braces mark a multivalued attribute; parentheses enclose the components of a composite attribute.

```
{JobHistory(CompanyName, JoinDate, EndDate,
  {Position(PositionName, AwardMonth, AwardYear)},
  {Salary(BasicPay, Allowances, DurationYears, Grade)}}}
```

Company/period	Positions	Salary entries
Acme, 2020-01 to 2022-12	Analyst (01,2020); Senior Analyst (07,2021)	(40000,2000,1.5,G1); (50000,3000,1.5,G2)

The two inner sets belong to the company-employment entry, not directly to all of an employee's history. They are independent sets; do not zip the first position to the first salary unless an additional alignment rule is supplied. BasicPay and Allowances use one declared currency/pay period in this example. The book does not provide salary effective dates, so chronological placement of every salary entry cannot be inferred from DurationYears alone.

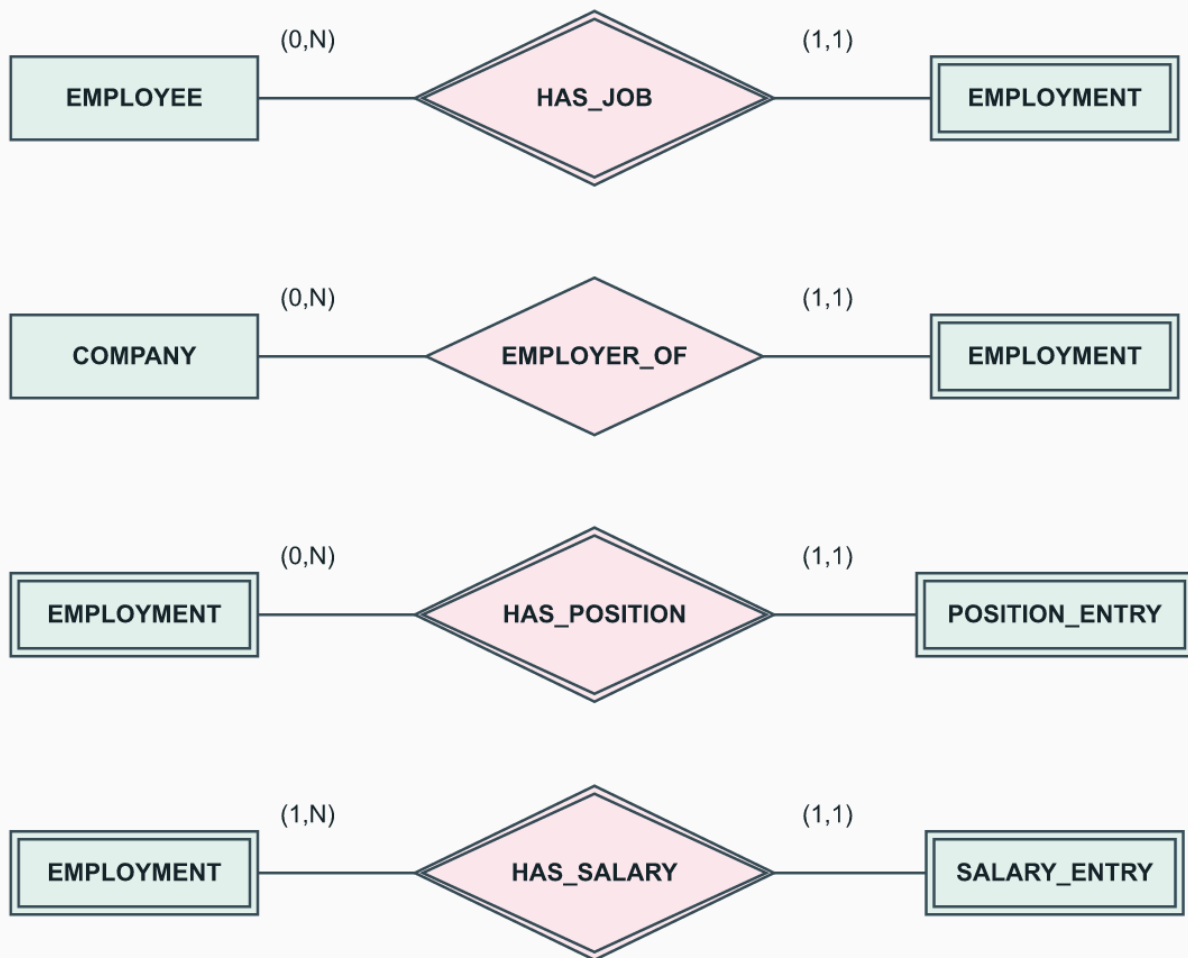
3.18. Job History as Entity Types

Source: exercise 3.18, p.127. One valid alternative to Answer 3.17 follows.

Type	Key/partial key	Other attributes
EMPLOYEE	EmployeeId	Name
COMPANY	CompanyId (assumed assigned ID)	CompanyName
EMPLOYMENT	EmploymentNo, partial within EMPLOYEE	JoinDate, EndDate
POSITION_ENTRY	PositionNo, partial within EMPLOYMENT	PositionName, AwardMonth, AwardYear
SALARY_ENTRY	SalaryNo, partial within EMPLOYMENT	BasicPay, Allowances, DurationYears, Grade

Job history preserves employment spells

ER: counts beside each entity



Counts belong to the entity beside each pair. See the accompanying keys and assumptions.

An employee can return to a company, so EmployeeId plus CompanyId alone is not enough for an employment spell. Full position identity is (EmployeeId, EmploymentNo, PositionNo); salary identity is analogous. Assume every employment spell records at least one salary entry; positions may be absent. Salaries remain attached to employment rather than to position: the prompt does not state a one-to-one position/salary correspondence. The artificial local numbers are a design choice, not values supplied by the book.

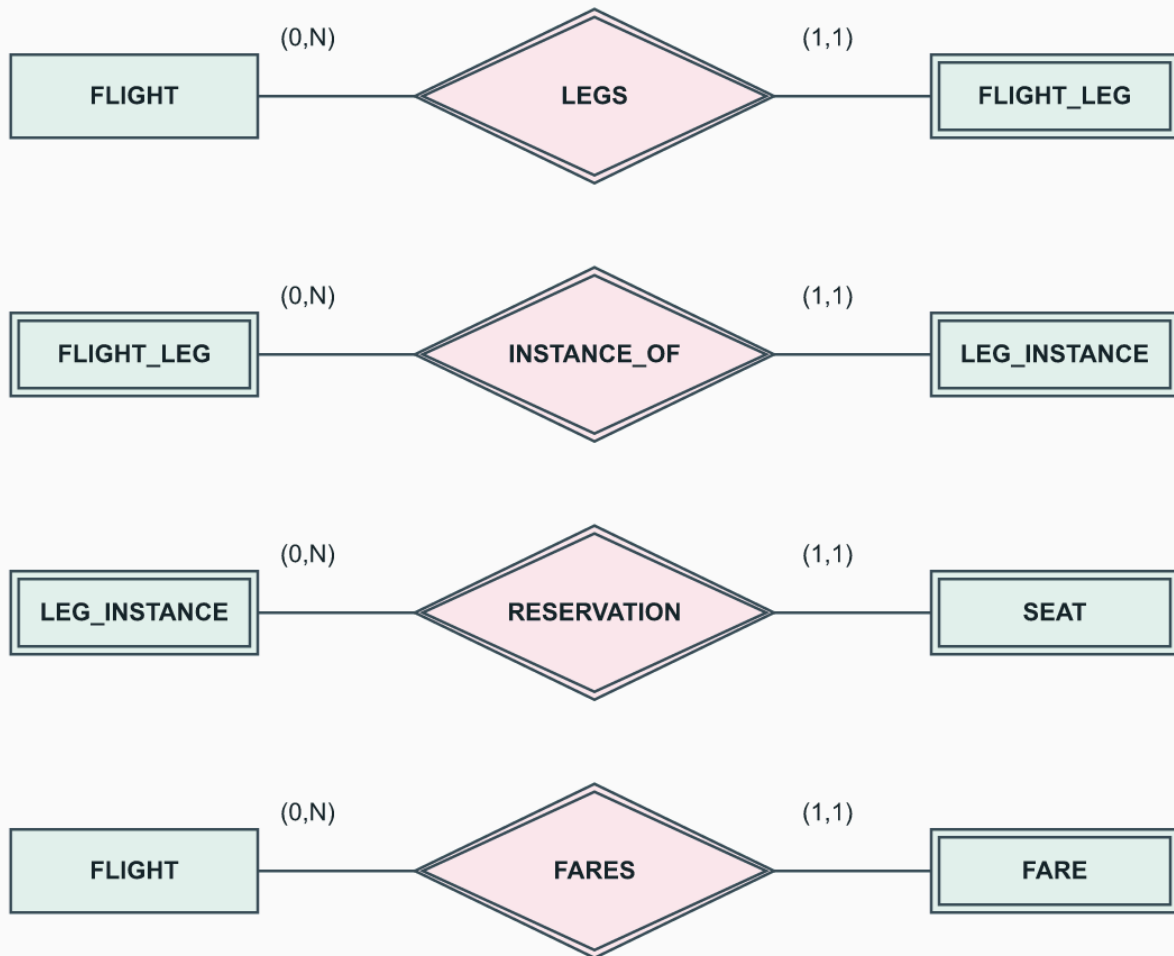
3.19. Reading the AIRLINE Schema

Source: exercise 3.19 and Figure 3.21, pp.127-128.

Type	Identity	Other displayed attributes
AIRPORT	Airport_code	Name, City, State
AIRPLANE_TYPE	Type_name	Company, Max_seats
AIRPLANE	Airplane_id	Total_no_of_seats
FLIGHT	Number	Airline, Weekdays
FLIGHT_LEG (weak)	Flight Number + Leg_no	Leg_no is partial
LEG_INSTANCE (weak)	Flight Number + Leg_no + Date	No_of_avail_seats
FARE (weak)	Flight Number + Code	Amount, Restrictions
SEAT (weak reservation record)	Leg-instance identity + Seat_no	Seat_no is partial

AIRLINE: identity follows the owner hierarchy

ER: counts beside each entity



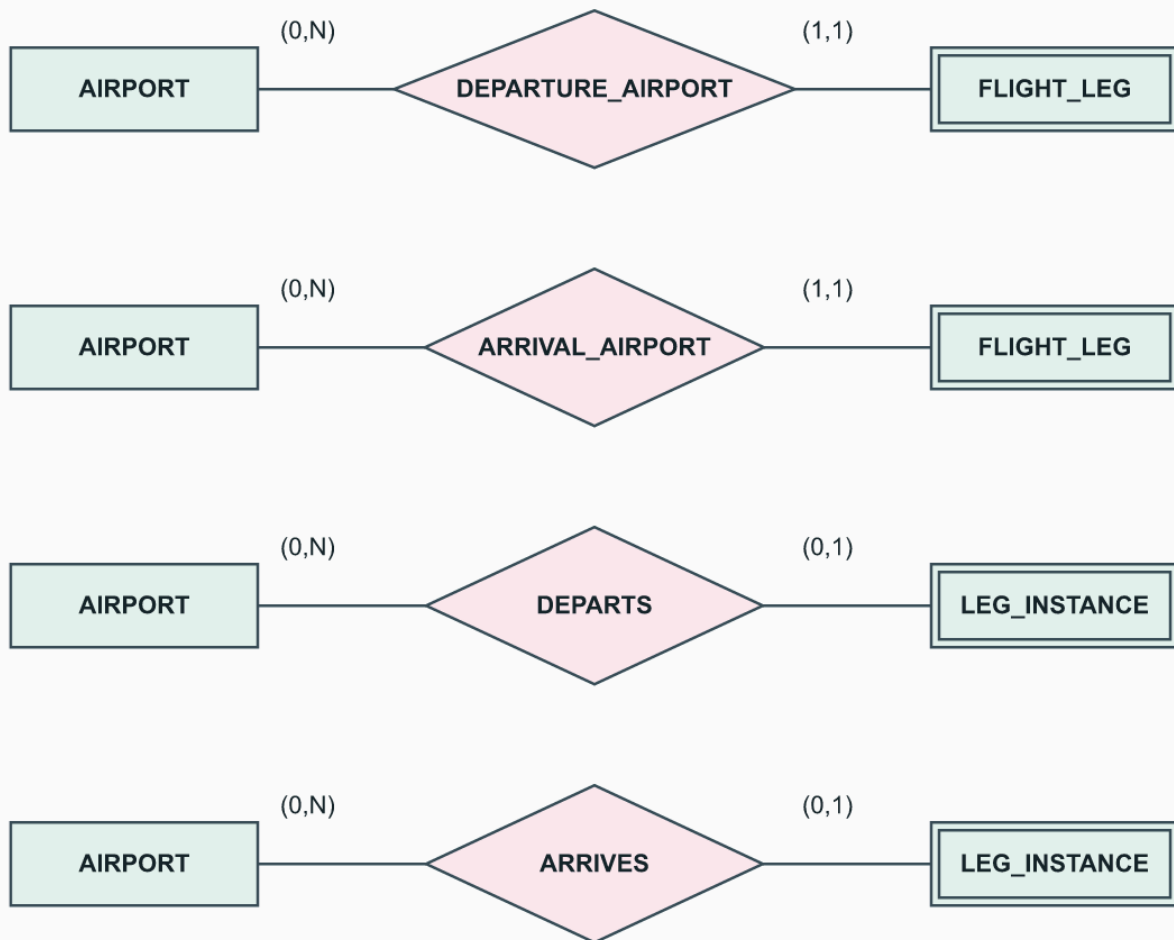
Counts belong to the entity beside each pair. See the accompanying keys and assumptions.

The following min-max pairs are translations of the photographed lines, not assumptions about what a commercial airline ought to require.

Relationship	First participant	Second participant	Relationship attributes
CAN_LAND	AIRPORT (0,N)	AIRPLANE_TYPE (0,N)	None
TYPE	AIRPLANE_TYPE (0,N)	AIRPLANE (1,1)	None
LEGS, identifying	FLIGHT (0,N)	FLIGHT_LEG (1,1)	None
DEPARTURE_AIRPORT	AIRPORT (0,N)	FLIGHT_LEG (1,1)	Scheduled_dep_time
ARRIVAL_AIRPORT	AIRPORT (0,N)	FLIGHT_LEG (1,1)	Scheduled_arr_time
INSTANCE_OF, identifying	FLIGHT_LEG (0,N)	LEG_INSTANCE (1,1)	None
ASSIGNED	AIRPLANE (0,N)	LEG_INSTANCE (1,1)	None
DEPARTS	AIRPORT (0,N)	LEG_INSTANCE (0,1)	Dep_time
ARRIVES	AIRPORT (0,N)	LEG_INSTANCE (0,1)	Arr_time
FARES, identifying	FLIGHT (0,N)	FARE (1,1)	None
RESERVATION, identifying	LEG_INSTANCE (0,N)	SEAT (1,1)	Customer_name, Cphone

AIRLINE: scheduled and actual events differ

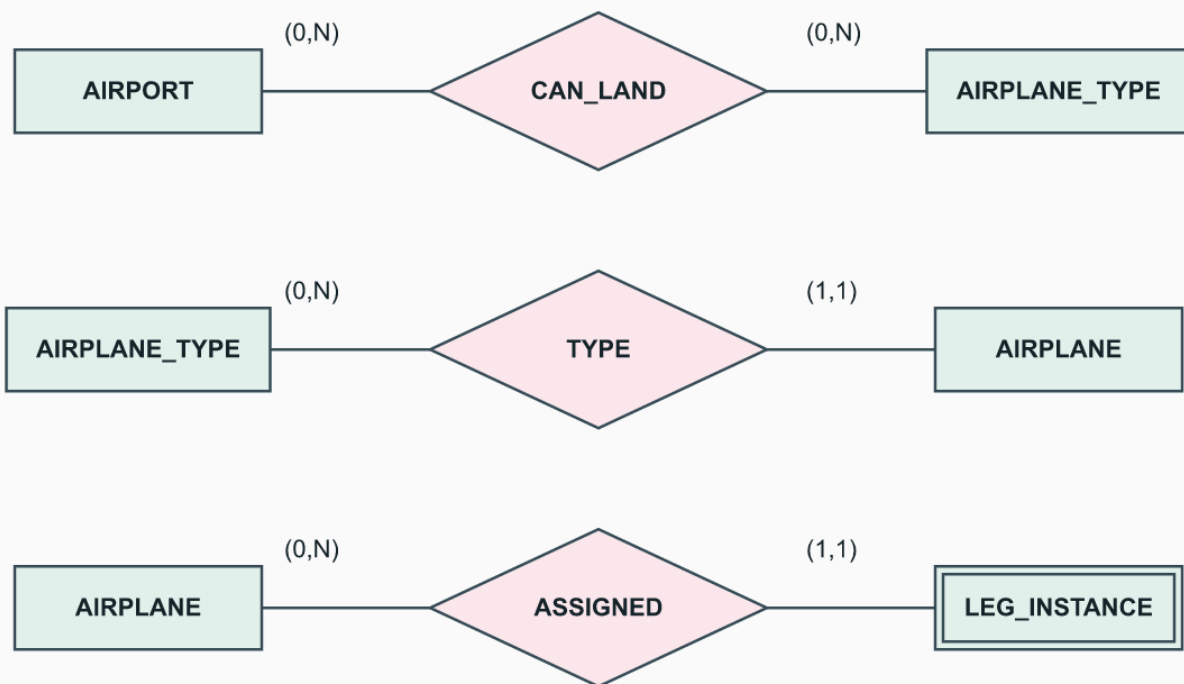
ER: counts beside each entity



Scheduled and actual times belong to the corresponding relationships.

AIRLINE: aircraft types and assignment

ER: counts beside each entity



Aircraft identity and type differ. Every dated leg instance has exactly one assigned airplane.

Important distinctions: scheduled airports belong to a flight leg, while actual departure/arrival belongs to its dated instance. The latter have single lines on the instance side, allowing a not-yet-departed/not-yet-arrived instance. Customer_name and Cphone are drawn on RESERVATION, not on an independent CUSTOMER entity. The arrow labeled Instances is explanatory, not an additional relationship.

The drawing does not enforce connecting-leg airport continuity, chronological times, aircraft availability, capacity consistency, seat-number validity, or that the assigned airplane's type can land at both airports. Those require additional rules; do not claim that cardinality symbols alone enforce them.

3.20. Modeling a Database Environment

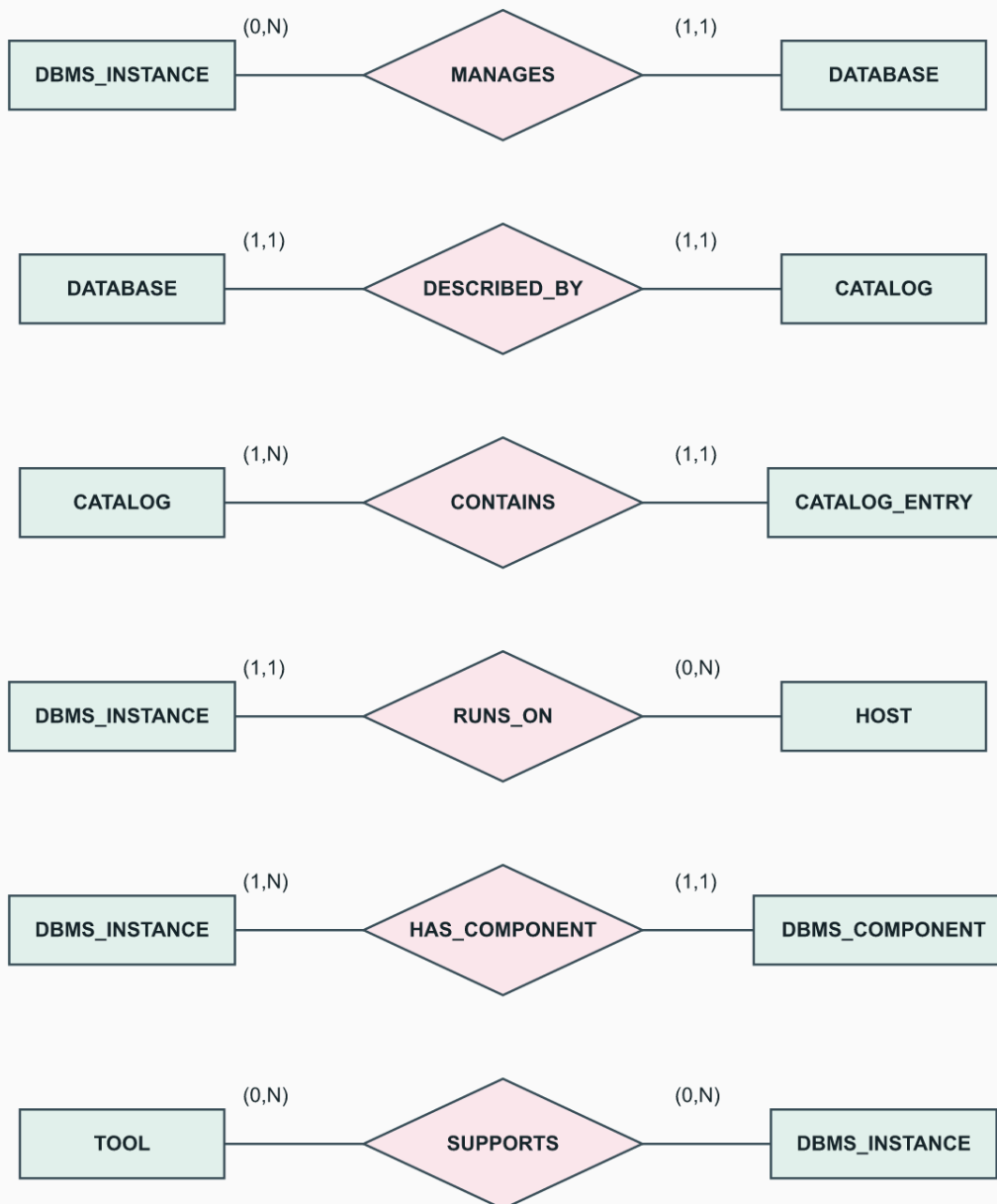
Source: exercise 3.20, p.127; Ch1, sections 1.4-1.5, pp.15-17; Ch2, sections 2.4.1-2.4.3, pp.42-46, for people, modules, catalogs, and tools. This is one ER design for an organization's database environment, not a unique schema supplied by the book.

The boundary includes managed databases, their catalog descriptions, running DBMS instances and components, hosts, tools, applications, accounts, and people. It does not model vendor employment, every hardware part, or internal query plans. All identifiers and cardinalities below are explicit design assumptions.

Entity	Assumed key	Attributes
DBMS_INSTANCE	InstanceId	Product, Version
DATABASE	Databaseld	Name
CATALOG	CatalogId	Name
CATALOG_ENTRY	EntryId	ObjectName, Kind, Definition
PERSON	PersonId	Name, Roles (multivalued)
USER_ACCOUNT	AccountId	LoginName
APPLICATION	AppId	Name
HOST	HostId	Name, OperatingSystem
DBMS_COMPONENT	ComponentId	Name, Function
TOOL	ToolId	Name, Purpose

Database environment: software, data, and hosts

ER: counts beside each entity

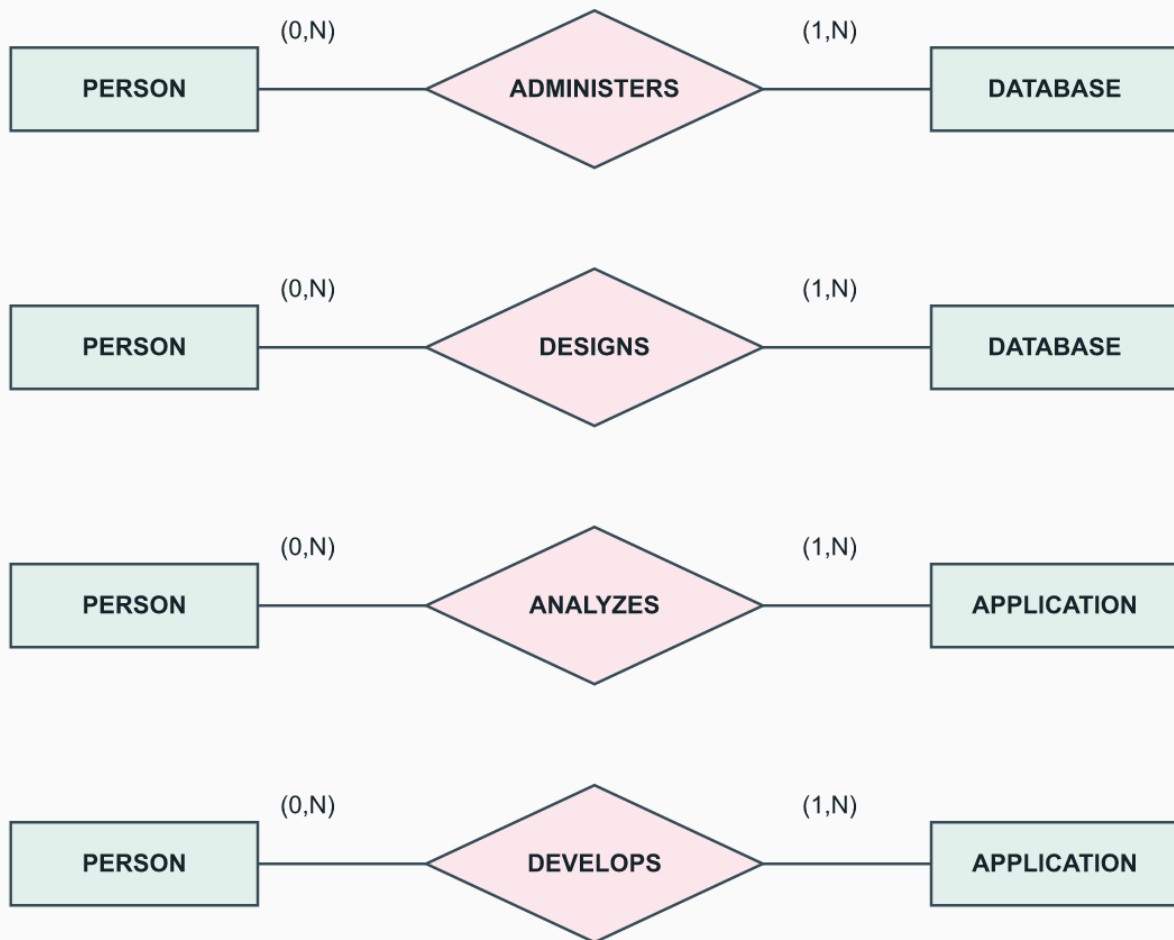


One possible organizational inventory. The accompanying text states the scope and all cardinality assumptions.

Each database has one managing instance and one catalog; each catalog describes one database and has at least one entry. Each entry describes one database object and belongs to one catalog. This per-database catalog is an explicit assumption. Each instance runs on one host and includes at least one component, such as a query processor or storage manager; an installed component belongs to one instance in this model. A tool can support multiple instances, and an instance can have multiple tools.

Database environment: people can hold several roles

ER: counts beside each entity

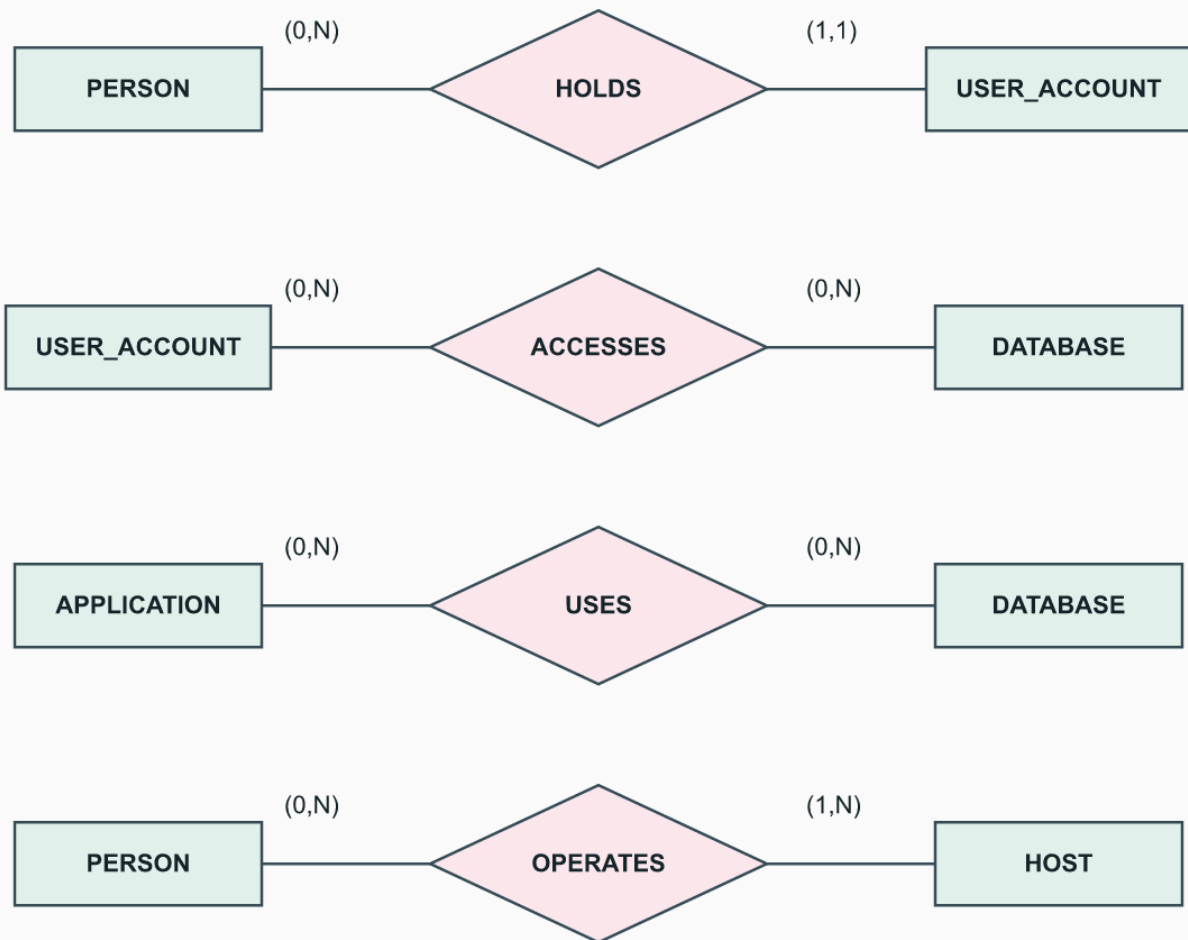


All PERSON rectangles denote the same type. Role eligibility is a separate constraint stated in the answer.

Each database has at least one administrator and one designer; each application has at least one analyst and developer. One person may hold several roles. The ADMINISTERS participant must have the DBA role, DESIGNS the database-designer role, ANALYZES the system-analyst role, and DEVELOPS the application-programmer role. These role checks are additional constraints, not consequences of min-max labels. End users can be casual, parametric, sophisticated, or standalone users, following Ch1; their access is represented in the next diagram.

Database environment: accounts, use, and operation

ER: counts beside each entity



Personal accounts and one-host DBMS instances are assumptions. These links do not implement access control.

Each personal account belongs to one person; a person can hold several accounts. Accounts and applications can access several databases. Each host has at least one operator, who can operate several hosts. PERSON.Roles can also record DBMS implementers and tool developers without incorrectly making all people end users. The model inventories these roles; it does not record their vendor development history.

For example, illustrative person P1 can both administer database D1 and design its schema, while P2 uses account U2 to access D1. Application A1 also uses D1; this schema does not record which application P2 used for a particular request. CATALOG_ENTRY E1 describes a table in D1; it is metadata, not one of that table's business records. Access links may carry privileges, but an ER diagram does not enforce authorization. DBMS_INSTANCE is an installed/running instance, not a product brand. Shared service accounts, replication, and distributed hosts would require revising these stated assumptions.

3.21. Congressional Voting Database

Source: exercise 3.21, pp.127-128. Model one stated two-year term only.

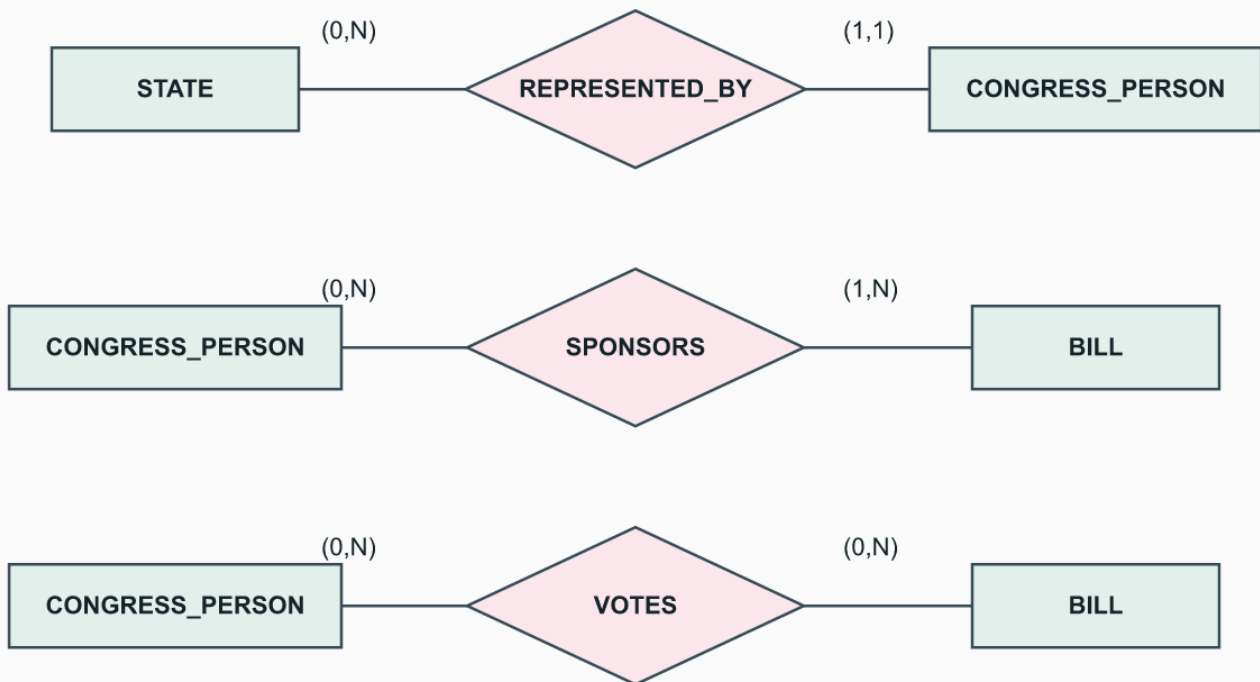
Type	Key	Attributes
STATE	Name	Region
CONGRESS_PERSON	MemberId (assumed)	Name, District, StartDate, Party
BILL	BillId (assumed)	BillName, VoteDate, PassedOrFailed

Attribute	Allowed values or meaning required by the exercise
STATE.Region	Northeast, Midwest, Southeast, Southwest, West
CONGRESS_PERSON.Party	Republican, Democrat, Independent, Other
CONGRESS_PERSON.StartDate	Date the member was first elected
BILL.PassedOrFailed	Yes or No: whether the bill passed
VOTES.Vote	Yes, No, Abstain, Absent

Member and bill IDs are added identifiers; the question does not guarantee that names alone are unique. District denotes the district represented within the state.

Voting is separate from sponsorship

ER: counts beside each entity



VOTES carries Vote. Complete-roll-call coverage is an additional rule, not a local count.

VOTES has Vote in {Yes, No, Abstain, Absent}. The member-bill pair identifies one vote in this simplified model. SPONSORS is separate; a member need not sponsor a bill to vote on it. Each member represents one state. Assume every recorded bill has at least one sponsor and one roll-call event.

Member	Bill	Vote	Sponsor?
M1	B1	Yes	Yes
M2	B1	Absent	No

Do not infer an Absent vote from a missing database row. A complete roll call requires a record for every eligible member; pairwise min-max constraints alone do not enforce this all-members rule. If bills have multiple votes, introduce VOTE_EVENT(BillId,EventNo,VoteDate) and attach votes to the event. Historical party or district changes require dated terms; they are outside this one-term design.

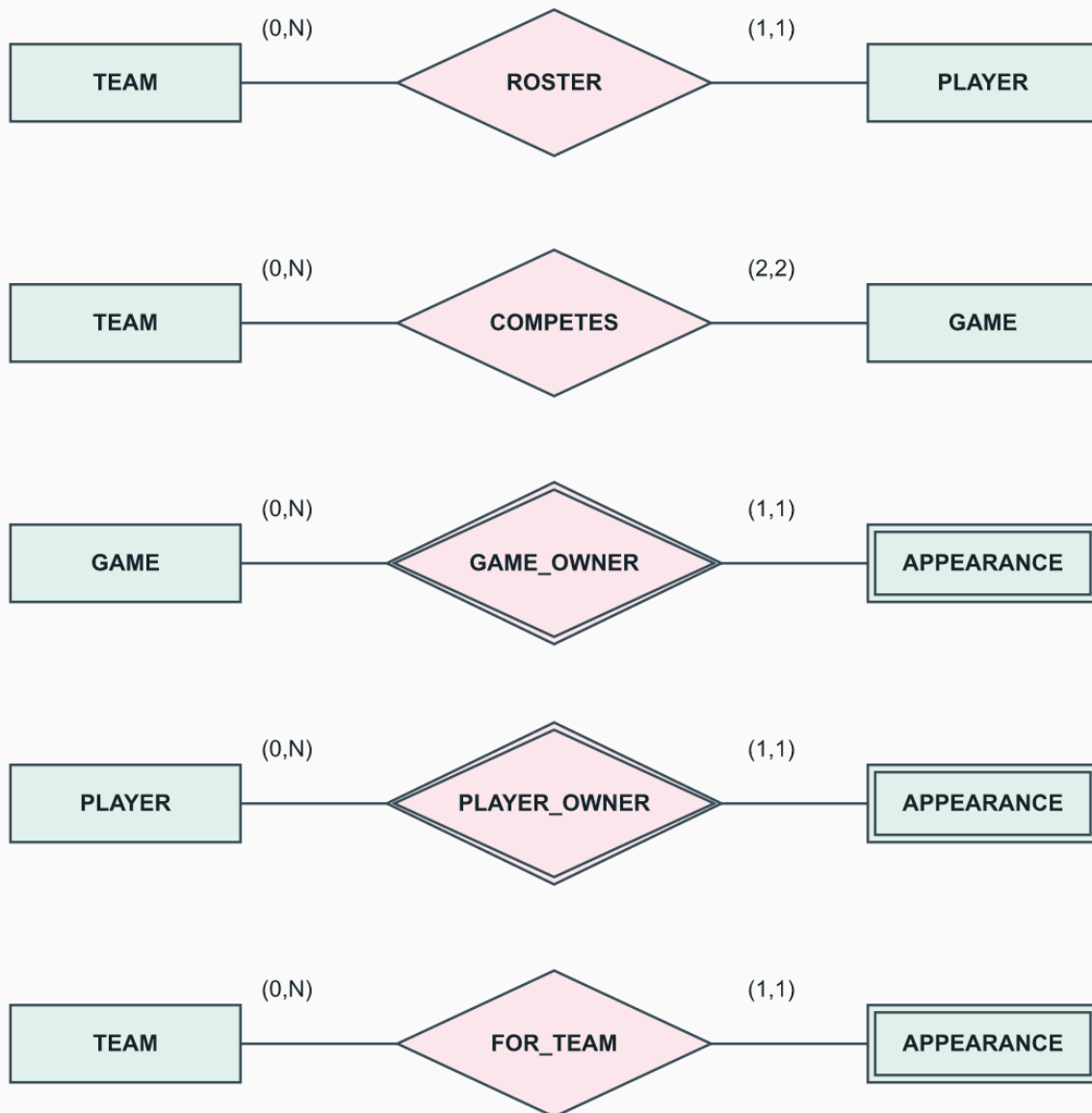
3.22. Teams, Games, and Player Participation

Source: exercise 3.22, pp.128-129. Use basketball for this illustrative design.

Type	Key	Attributes
TEAM	TeamId	Name
PLAYER	PlayerId	Name
GAME	GameId	Date, Venue
APPEARANCE (weak)	Game + Player	Positions (multivalued)

A game appearance has a player and a team

ER: counts beside each entity



GAME and **PLAYER** jointly identify **APPEARANCE**, with no partial key. Positions belong to **APPEARANCE**; Score/Result belong to **COMPETES**.

Assume a season-long roster with each player on one team. **COMPETES** relates a team to a game and carries Score and Result. Each game has exactly two teams. An **APPEARANCE** is identified by a game and player; **FOR_TEAM** records the team represented in that game. Additional constraints require that team to participate in the game and match the player's roster. Team transfers need dated rosters.

Game	Team	Score	Result
G1	T1	90	Win
G1	T2	82	Loss

Game	Player	Team	Positions played
G1	P1	T1	Guard, Forward
G1	P2	T2	Center

Roster member P3 may have no G1 appearance. Positions belong to the game-specific appearance, not permanently to PLAYER. This records the requested set of positions; timed substitutions would need additional segments.

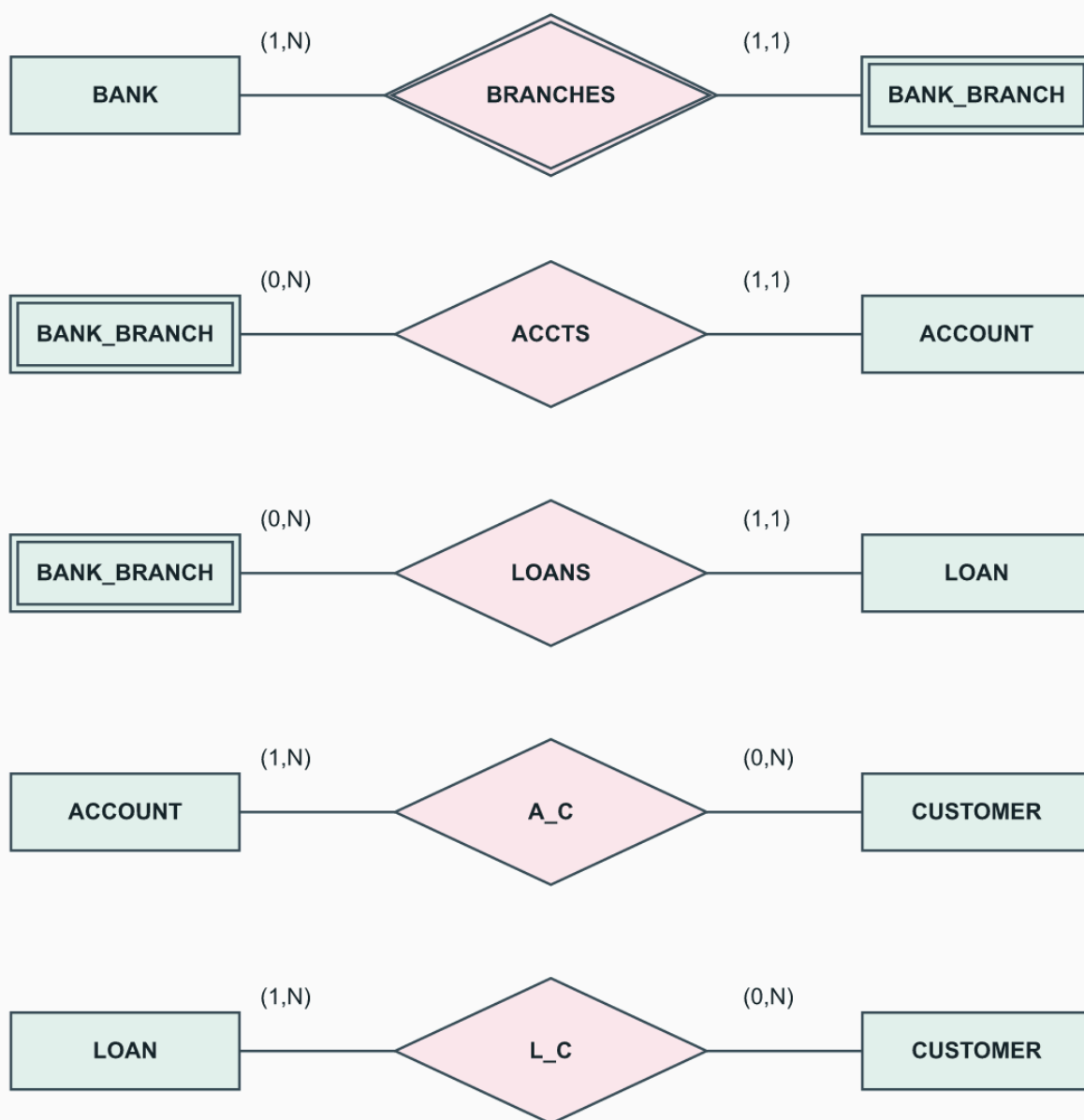
3.23. BANK Schema

Source: exercise 3.23, pp.129-130, Figure 3.22.

Part	Answer
a	Regular types: BANK, ACCOUNT, LOAN, CUSTOMER
b	Weak type: BANK_BRANCH; partial key Branch_no; identifying relationship BRANCHES
c	Each branch has exactly one bank; Branch_no is unique within that bank, not necessarily across banks.

BANK: distinguish branch identity from account identity

ER: counts beside each entity



Counts belong to the entity beside each pair. See the accompanying keys and assumptions.

Part d: relationship	First participant	Second participant
BRANCHES	BANK (1,N)	BANK_BRANCH (1,1)
ACCTS	BANK_BRANCH (0,N)	ACCOUNT (1,1)
LOANS	BANK_BRANCH (0,N)	LOAN (1,1)
A_C	ACCOUNT (1,N)	CUSTOMER (0,N)
L_C	LOAN (1,N)	CUSTOMER (0,N)

For part d, the double lines give minimum one for BANK in BRANCHES, BANK_BRANCH in BRANCHES, ACCOUNT in ACCTS/A_C, and LOAN in LOANS/L_C. The other displayed participations have single lines, giving minimum zero. The 1:N ratios make each branch belong to one bank and each account/loan to one branch. A_C and L_C are M:N, allowing several customers per account/loan and several accounts/loans per customer. These marks justify each table entry; the identifying diamond alone does not impose a minimum of one branch per bank.

Part e: banks have Code/Name/Addr; branches have local Branch_no/Addr. Every bank has at least one branch. Each account or loan belongs to one branch and has at least one customer. Customers can have several accounts/loans or none. Accounts have globally identifying Acct_no plus Balance/Type; loans have Loan_no plus Amount/Type; customers have Ssn plus Name/Phone/Addr. Joint accounts and joint loans are permitted. Those global account/loan keys are not partial keys.

Part f change	Updated constraint
Each customer needs an account	CUSTOMER in A_C becomes (1,N).
Customer has at most two loans	CUSTOMER in L_C becomes (0,2).
Branch has at most 1,000 loans	BANK_BRANCH in LOANS becomes (0,1000).

Other minima and maxima remain unchanged. For example, limiting a branch's loans does not cap the number of customers on one loan.

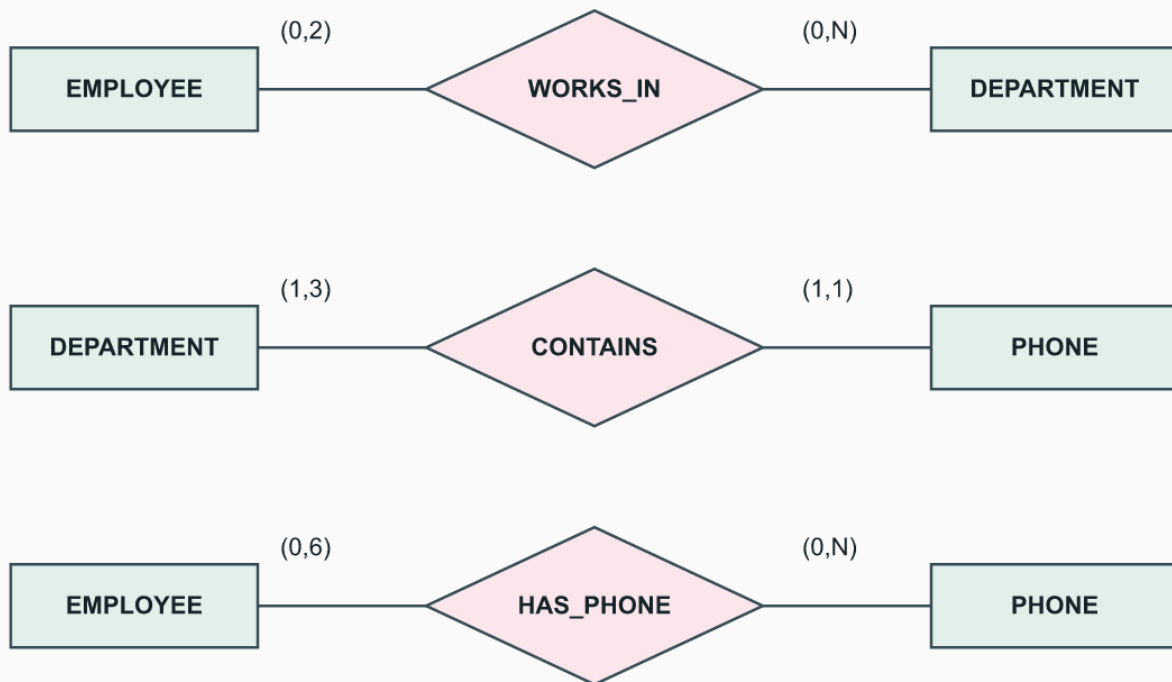
3.24. Employees, Departments, and Phones

Source: exercise 3.24, p.130, Figure 3.23.

Relationship	First participant	Second participant	Basis
WORKS_IN	EMPLOYEE (0,2)	DEPARTMENT (0,N)	Employee bound given; department bound assumed
CONTAINS	DEPARTMENT (1,3)	PHONE (1,1)	Department bound given; one owner department per phone assumed
HAS_PHONE	EMPLOYEE (0,6)	PHONE (0,N)	Under the derivation rule below

Employee phones derived under a declared rule

ER: counts beside each entity



HAS_PHONE is redundant only if it equals the composition of WORKS_IN and CONTAINS.

For this solution, HAS_PHONE(*e*,*p*) holds exactly when there exists a department *d* such that WORKS_IN(*e*,*d*) and CONTAINS(*d*,*p*). That equality makes HAS_PHONE redundant. An employee in two departments, each with three distinct phones, then has six phone links at most. Neither employee nor phone participation is required.

This derivation is an additional assumption, not supplied by the bare triangle. If HAS_PHONE instead means an employee's individually assigned handset, shared department membership cannot determine it and the relationship must be retained.

Facts	Derived employee-phone pairs
E1 works in D1,D2; D1 contains P1; D2 contains P2,P3	(E1,P1), (E1,P2), (E1,P3)

3.25. Courses, Instructors, and Texts

Source: exercise 3.25, p.130, Figure 3.24.

Relationship	First participant	Second participant
TEACHES	INSTRUCTOR (2,4)	COURSE (1,N), assumed at least one teacher
USES	COURSE (0,5)	TEXT (1,N)

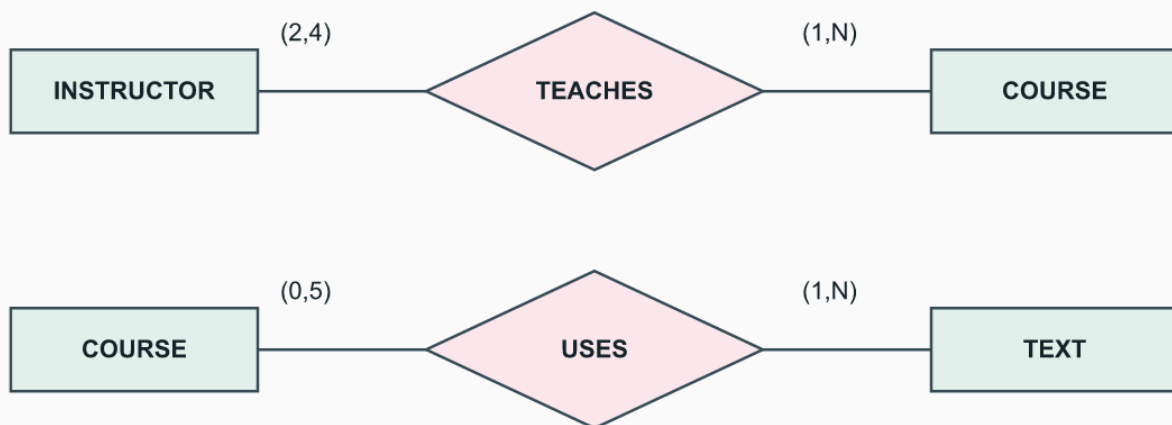
If different instructors use different texts for different courses, ADOPTS must retain all three participants: INSTRUCTOR, COURSE, TEXT.

Instructor	Course	Adopted text
I1	C1	T1
I1	C2	T2

Binary instructor-text links lose which course uses which text. They work only with an additional rule that the instructor's choice does not depend on course.

Course use and instructor teaching are different

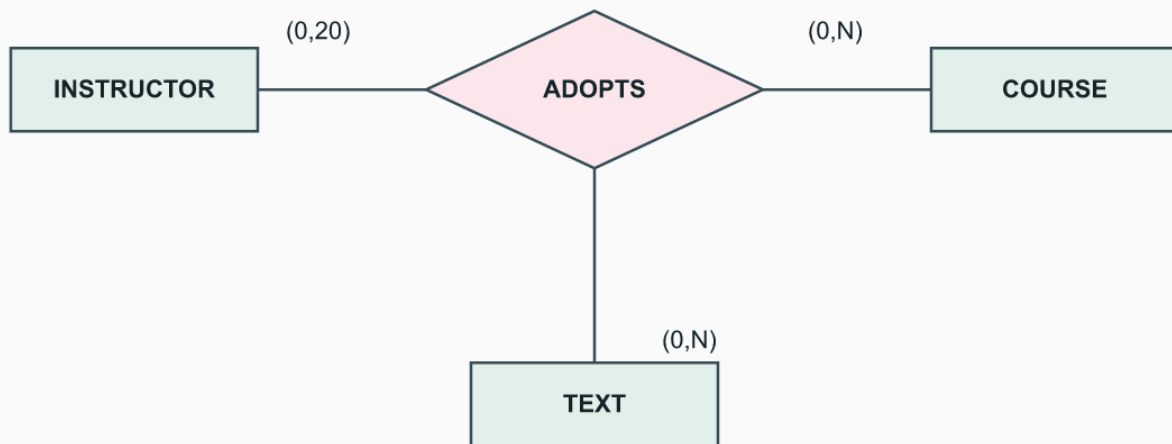
ER: counts beside each entity



ADOPTS needs instructor, course, and text together when choices vary by course. See the ternary constraints in the answer.

ADOPTS: an instructor chooses a text for a course

ER: min-max counts of ternary instances



Each triple must satisfy TEACHES and USES. The 20 bound is four courses times five texts; course/text totals have no fixed upper bound.

Possible state	ADOPTS triples
A	(I1,C1,T1), (I1,C2,T2)
B	(I1,C1,T2), (I1,C2,T1)

In both states I1 teaches C1 and C2, and both courses permit T1 and T2 in USES. The binary instructor-text set is $\{(I1,T1), (I1,T2)\}$ in both states. It cannot tell whether C1 uses T1 or T2 through I1. The ternary triples preserve that difference.

Assume ADOPTS links only an instructor who teaches that course and a text used by it. For the ternary diagram use instructor (0,20), course (0,N), and text (0,N). The instructor upper bound follows from at most four courses times at most five distinct texts per course. It does NOT mean five ADOPTS instances per course: several instructors may adopt the same text. Add fixed-pair constraints: at most five distinct texts per instructor-course pair, and at most four distinct courses per instructor. If USES must equal the projection of ADOPTS, each used text/course needs at least one adopting instructor; this is an additional rule.

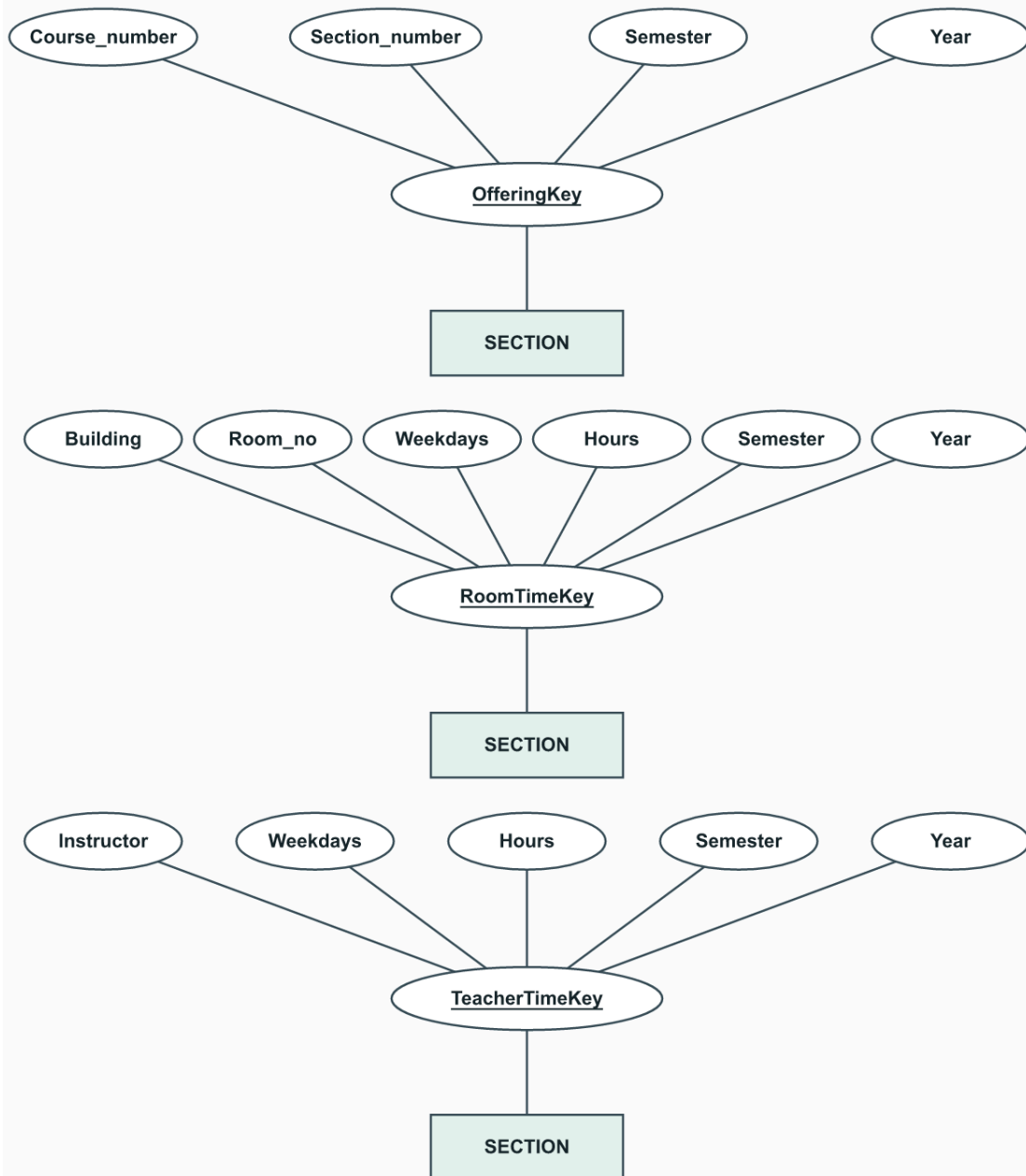
3.26. Composite SECTION Keys

Source: exercise 3.26, p.131. This exercise's flat attribute list differs from the SecId-based example in 3.10.

Proposed key	Additional business rule needed
(Course_number, Section_number, Semester, Year)	Local section number is unique within course and term, as stated.
(Building, Room_no, Weekdays, Hours, Semester, Year)	No two sections use the same room/time in a term.
(Instructor, Weekdays, Hours, Semester, Year)	One instructor cannot teach two sections at that time; no combined sections.

Three possible composite keys for SECTION

ER: assumptions in Answer 3.26



Underline each composite key, not its separate components. The last two keys require additional scheduling rules.

Use three composite attributes and underline each composite name. Share components where necessary rather than claiming each component is independently unique. All three combinations require minimality under the stated assumptions. The last two are plausible extra rules, not logical consequences of local section numbering alone. Instructor must identify an instructor, not an ambiguous name. Identical patterns and overlapping but unequal time patterns need separate checks.

3.27. Plausible Cardinality Ratios

Source: exercise 3.27, p.131. Ratios below are from Entity 1 to Entity 2. This is a modeling exercise, not a claim about every institution or jurisdiction.

Pair	Ratio	Assumption and possible variation
STUDENT : SOCIAL_SECURITY_CARD	1:1	One current card per student and one holder per card; replacement history changes this.
STUDENT : TEACHER	M:N	Students take instruction from multiple teachers.
CLASSROOM : WALL	M:N	A physical wall can bound two rooms; room-specific wall surfaces would give 1:N.
COUNTRY : CURRENT_PRESIDENT	1:1	One current president in the chosen political model; some countries have none or different institutions.
COURSE : TEXTBOOK	M:N	Courses can use several books and books can be reused.
ITEM : ORDER	M:N	ITEM means product/catalog type, not an individual order line.
STUDENT : CLASS	M:N	Students enroll in multiple class offerings.
CLASS : INSTRUCTOR	N:1	One instructor per class; team teaching would be M:N.
INSTRUCTOR : OFFICE	N:1	Each instructor has one office; offices may be shared. Multiple offices would change this.
EBAY_AUCTION_ITEM : EBAY_BID	1:N	One auction listing receives many bids; each bid belongs to one listing.

For ITEM interpreted as an order-line occurrence, the ratio becomes N:1. Maximum ratios do not determine optionality; a country without a president requires an appropriate minimum constraint or a different model.

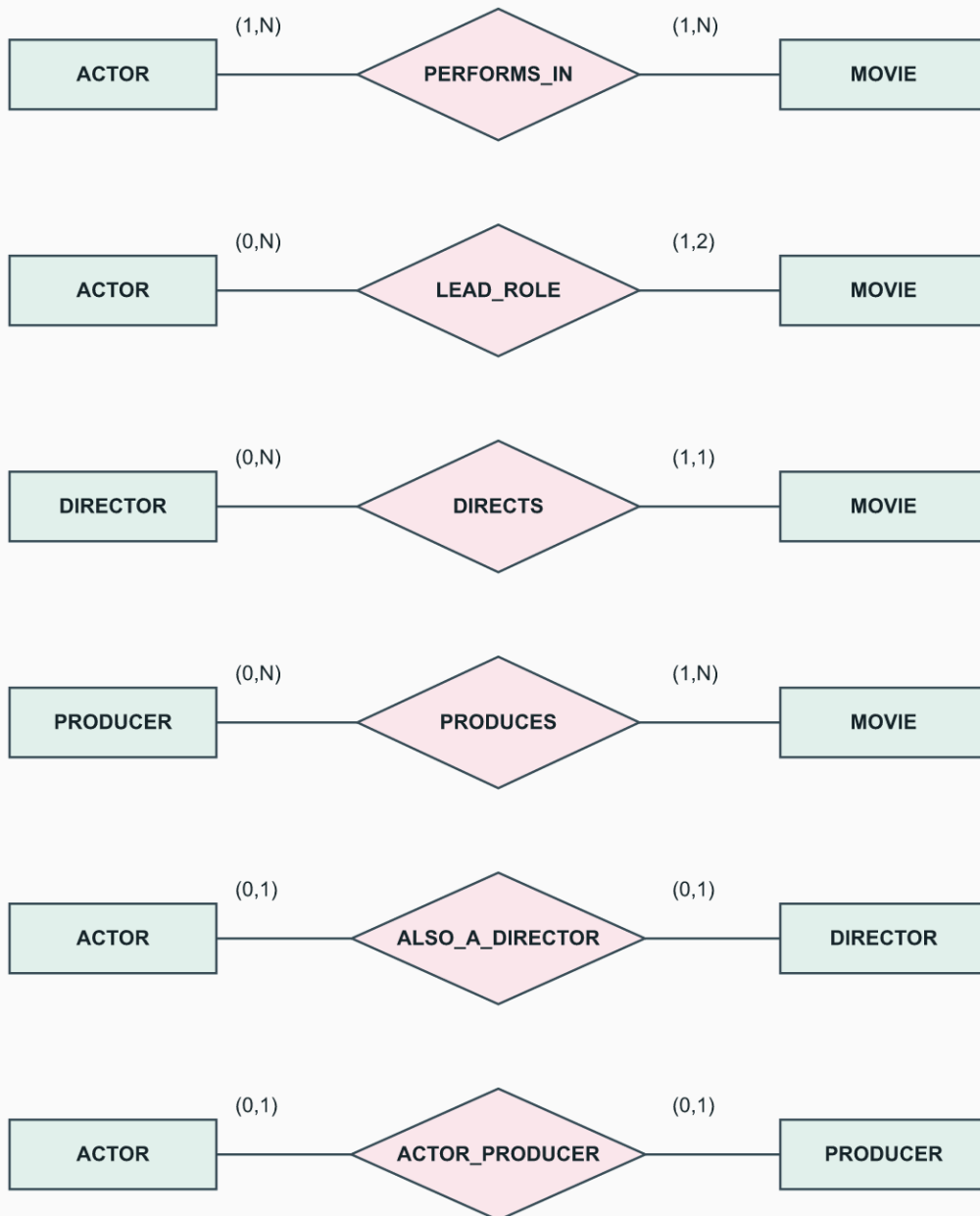
3.28. MOVIES: True, False, or Maybe

Source: exercise 3.28, pp.131-132, Figure 3.25. True means required/permitted as phrased by the schema; False contradicts its constraints; Maybe describes a state-dependent assertion not forced by the schema. Distinguish "can/cannot" (permission) from "there are" (actual existence).

Part	Answer	Reason
a	True	ACTOR participates totally in PERFORMS_IN, so every represented actor has a movie.
b	Maybe	There is no maximum of ten, but no actor is required to exceed ten.
c	Maybe	Multiple lead roles are permitted, but need not occur.
d	True	The 2 on the actor side of LEAD_ROLE limits each movie to two lead actors.
e	Maybe	ALSO_A_DIRECTOR permits overlap but does not require every director to be an actor.
f	Maybe	ACTOR_PRODUCER is optional; its current set could be empty or nonempty.
g	False	Nothing forbids a producer also being an actor in another movie.
h	Maybe	A cast over twelve is allowed, not required.
i	Maybe	A producer may also be a director; no disjointness is specified.
j	Maybe	Each movie has exactly one director, but the schema says nothing about how many movies have exactly one producer.
k	Maybe	One director and several producers is legal, but its actual occurrence is not required.
l	Maybe	A person can act, lead, direct, and produce the same movie; no constraint forces such a person to exist.
m	Maybe	A director acting in their own movie is permitted, not required or forbidden.

MOVIES: the schema constrains, but does not enumerate, reality

ER: counts beside each entity

**Optional overlap does not imply disjointness.**

The following table answers each state-dependent statement with both a satisfying state and a counterexample. All records here are invented. Unless a row changes the cast or producers, give each movie one actor who also leads, one director, and one producer. People in different roles are distinct unless overlap is stated.

Part	A legal state where the statement is true	A legal state where it is false
b	One actor performs in eleven movies.	One movie with one actor.
c	One actor leads two movies.	One movie with one lead actor.
e	One person is the only actor, director, and producer.	The only director is distinct from the only actor.
f	The only producer is distinct from the only actor.	One person is both the producer and the actor.
h	One movie has thirteen performers, only one of whom is its lead.	One movie has one performer.
i	One person directs and produces a movie.	The only producer and director are distinct.
j	The only movie has one director and one producer: all movies, hence most, qualify.	The only movie has one director and two producers: none qualify.
k	One movie has one director and two producers.	The only movie has one producer.
l	One person acts, leads, directs, and produces one movie.	The actor, director, and producer are three distinct people.
m	The director and every performer are distinct.	A movie's director also performs in that movie.

Notice the negative wording in f and m: a state without overlap makes those statements true, while an overlapping state makes them false. In l, performing the roles in the same movie is a sufficient example; the question does not require all roles to concern the same movie. For j, these examples avoid any uncertainty about the boundary of "most": the fractions are 100% and 0%.

For g, let person X produce M1 and perform in M2. Supply both movies with their required leads, directors, and producers. This legal state directly refutes the claim that such a cross-movie combination is forbidden. For a, an actor with no performance violates total participation. For d, a third lead actor on one movie violates the maximum of two; one or two leads are permitted.

These examples establish both possibilities for every Maybe answer. Merely failing to find a sentence forbidding something would not establish them.

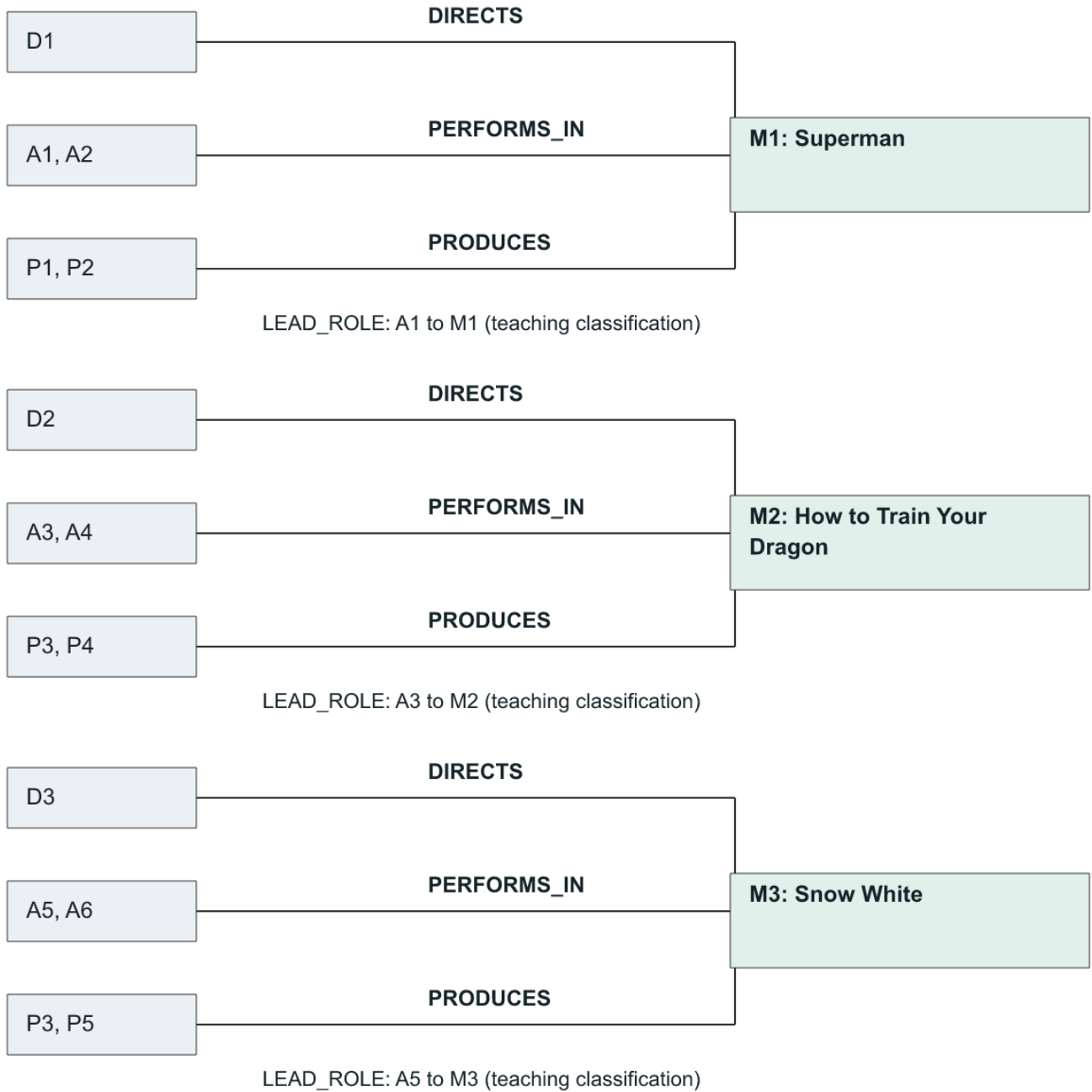
Figure 3.25 does not draw a separate producer-director overlap relationship. That omission is not a disjointness constraint. LEAD_ROLE being a subset of PERFORMS_IN is a natural additional semantic rule; the witnesses here satisfy it.

3.29. A Three-Movie Instance

Source: exercise 3.29, p.132. This is a deliberately selected, incomplete credits database for three 2025 releases, not a complete filmography. IDs below are invented; movie titles, the selected performers, directors, and producer credits are sourced.

Movie ID	Title/year	Director	Selected performers	Selected producers
M1	Superman (2025)	D1 James Gunn	A1 David Corenswet; A2 Rachel Brosnahan	P1 James Gunn; P2 Peter Safran
M2	How to Train Your Dragon (2025)	D2 Dean DeBlois	A3 Mason Thames; A4 Nico Parker	P3 Marc Platt; P4 Adam Siegel
M3	Snow White (2025)	D3 Marc Webb	A5 Rachel Zegler; A6 Gal Gadot	P3 Marc Platt; P5 Jared LeBoff

Sources checked for these selected credits: [Warner Bros./DC](#), [Universal cast/director](#), [Universal final release credits](#), [Disney](#). Omission of a credit is not evidence the person did not work on the film.

MOVIES: selected objects and relationship instances

A comma lists separate objects: each has its own link. Repeated P3 is the same producer. Full names appear in the table.

Relationship set	Instance pairs
DIRECTS	(D1,M1), (D2,M2), (D3,M3)
PERFORMS_IN	(A1,M1), (A2,M1), (A3,M2), (A4,M2), (A5,M3), (A6,M3)
PRODUCES	(P1,M1), (P2,M1), (P3,M2), (P4,M2), (P3,M3), (P5,M3)
LEAD_ROLE, teaching classification	(A1,M1), (A3,M2), (A5,M3)
ACTOR_PRODUCER	Empty within these selected actor and producer sets
ALSO_A_DIRECTOR	Empty within these selected actor and director sets

The performers playing Superman, Hiccup, and Snow White are classified as leads for this small teaching instance, based on the official character descriptions and story summaries. Hiccup is a central character, not a title character. This is not an exhaustive official lead-credit category. Each movie has one selected lead, satisfying Figure 3.25's 1-to-2 bound. Producer entries use the selected ordinary producer credits, not an exhaustive list of executive producers. James Gunn occurs in both DIRECTOR and PRODUCER; those rows denote the same person even though the diagram does not include a producer-director relationship. P3 appears in two PRODUCES instances. The empty actor-overlap sets apply only to the represented objects, not to these people's entire careers.

3.30. UNIVERSITY in UML

Source: exercise 3.30, p.133; Figure 3.20 and 3.8. The question points to exercise 3.16, which supplies extra UNIVERSITY constraints, not a complete class inventory. Use the UNIVERSITY schema in 3.10 as the base.

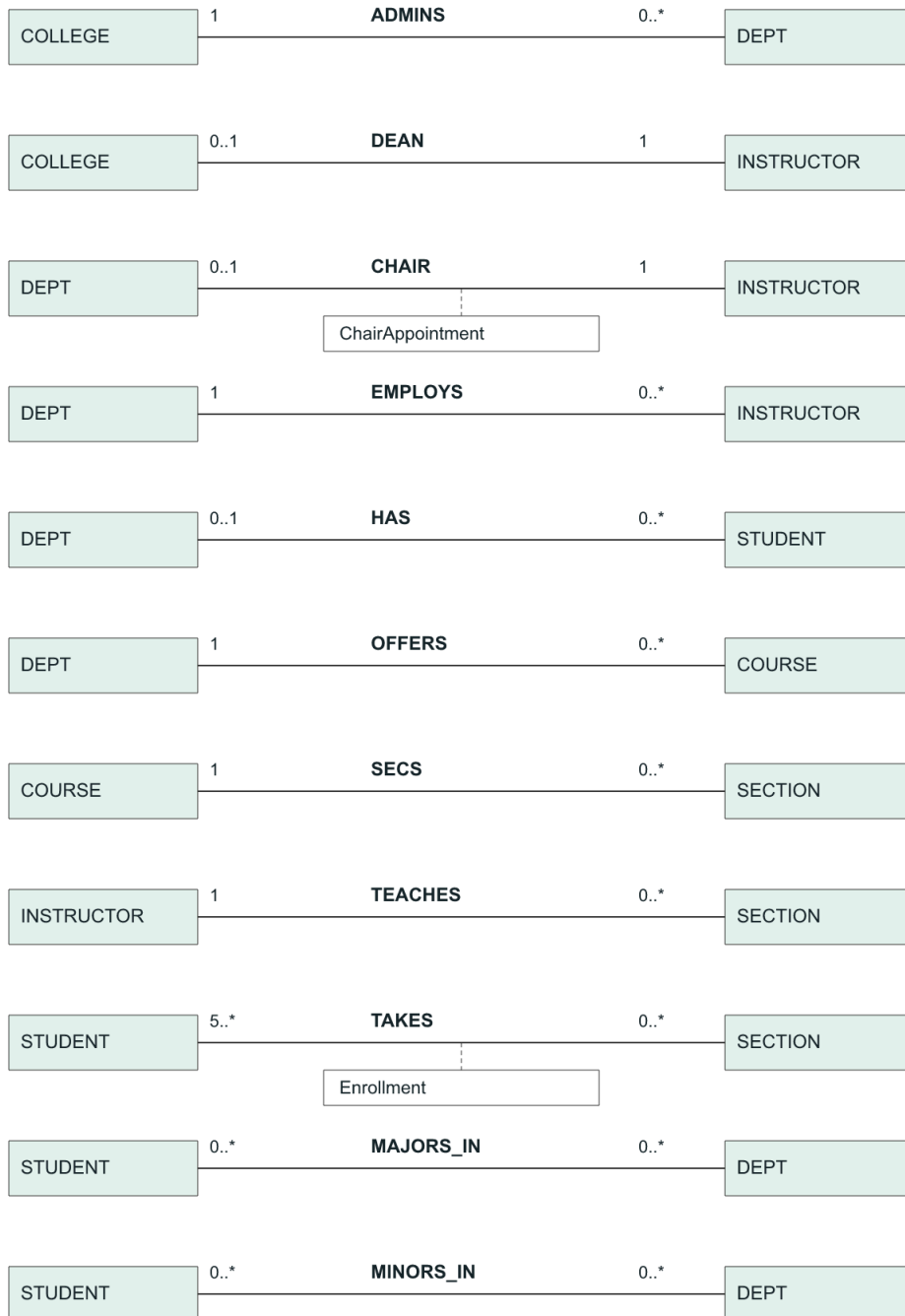
UNIVERSITY: UML class compartments

Student Sid; SName; Addr; Phone; DOB + computeGPA() + addMajor(); dropMajor() + addMinor(); dropMinor()	Department DCode; DName; DOffice; DPhone + addCourse(); dropCourse() + hireInstructor() + terminateInstructor()
Instructor Id; IName; IOffice; IPhone; Rank + assignGrade() + changeGrade()	Course CCode; CoName; Level; Credits; CDesc
Section SecId; SecNo; Sem; Year; CRoom DaysTime 	College CName; COffice; CPhone
Enrollment Grade 	ChairAppointment CStartDate

Compartments: class name, properties, operations. Empty operation compartments mean none specified by the exercise. Association links follow.

Class	Attributes (summary)	Operations required by this exercise
Student	Sid, SName, Addr, Phone, DOB	computeGPA(), addMajor(), dropMajor(), addMinor(), dropMinor()
Department	DCode, DName, DOffice, DPhone	addCourse(), dropCourse(), hireInstructor(), terminateInstructor()
Instructor	Id, IName, IOffice, IPhone, Rank	assignGrade(), changeGrade()
Course	CCode, CoName, Level, Credits, CDesc	None specified
Section	SecId, SecNo, Sem, Year, CRoom, DaysTime	None specified
College	CName, COffice, CPhone	None specified
Enrollment (association class)	Grade	May implement grade storage
ChairAppointment (association class)	CStartDate	None specified

Use associations for ADMINIS, DEAN, CHAIR, EMPLOYIS, HAS, OFFERS, SECS, TEACHES, and TAKES. Their UML end multiplicities are in the next figure; unlike the ER min-max rows elsewhere, each UML label counts objects at that end.

UML end multiplicities: read objects at the labeled end

MAJORS_IN and MINORS_IN are exercise extensions. Dashed links attach association classes; their properties appear in the class diagram.

Examples: a Section has one Course, so UML puts 1 at the Course end and 0..* at the Section end. A Section has at least five Students, so TAKES puts 5..* at the Student end and 0..* at the Section end. Use Enrollment on TAKES. For HAS, preserve Figure 3.20's 0..1 primary department per student and label the prose alternative of exactly one.

Multiple majors/minors require more than the original scalar Major attribute. For this solution add MAJORS_IN and MINORS_IN associations to Department, with 0..* at both ends; these are explicit exercise extensions, not replacements for primary-department HAS. Remove the original scalar Major to avoid duplicate storage, or clearly mark it derived if a primary-major label is retained. Course deletion and instructor termination must check existing links; they do not silently erase grades or historical sections.

For an illustrative GPA calculation, declare a grade-to-points map and compute $\text{sum}(\text{Credits} * \text{GradePoints}) / \text{sum}(\text{Credits})$ for included graded enrollments. With 3 credits at 4 points and 2 credits at 3 points, $\text{GPA} = 18/5 = 3.6$. Repeats, withdrawals, ungraded work, and a zero denominator need explicit policy. This example does not set the institution's GPA or permission rules. An instructor's grade operation must check that they are authorized for the target section.

Part	Example operation inputs	Required effect or check
a	Student S1; department D1; graded enrollments	add/dropMajor and add/dropMinor update the corresponding association, not the student's identity. computeGPA reads grades and course credits under a declared policy.
b	Department D1; course C1 or instructor I1	add/dropCourse changes OFFERS; hire/terminateInstructor changes EMPLOYS. Check existing sections, chair/dean links, and history before removal.
c	Instructor I1; student S1; section Q1; grade	assignGrade sets the Grade of TAKES(S1,Q1); changeGrade replaces that value. Check enrollment, authorization, and the one-current-grade rule.

Retain 3.16's one-instructor-per-section and student schedule constraints; adding operations does not relax them. The exercise asks for a class model, not an implementation of institutional policy or a working student information system.

3.31. UNIVERSITY Lab

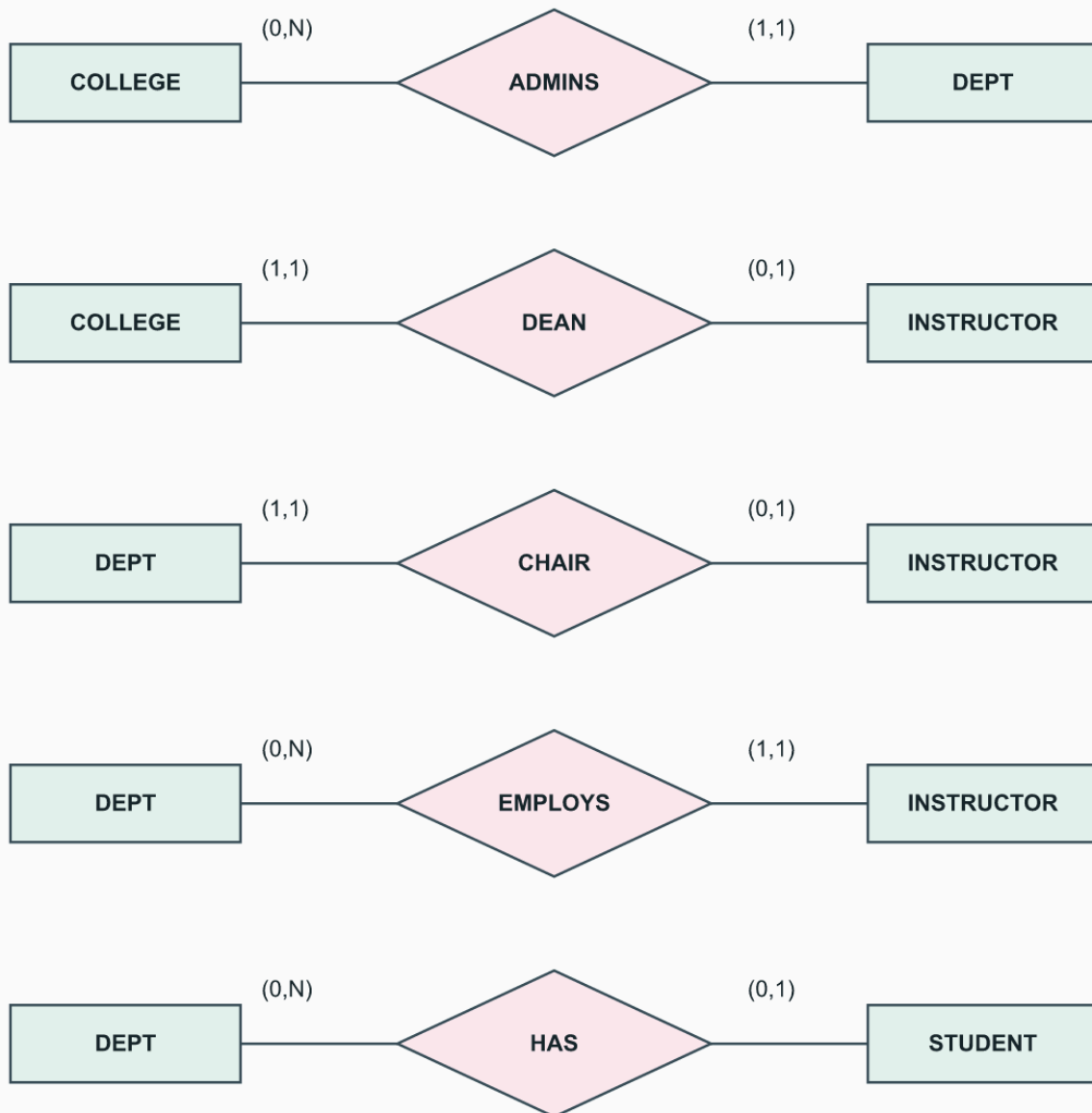
Source: laboratory exercise 3.31, p.133.

Completion: conceptual answer supplied; required modeling-application work not completed. No ERwin/Rational Rose model file or tool execution is supplied.

Use these original ER drawings, together with the key/attribute inventory below, as a complete conceptual design. This reconstructs 3.10 and adds the constraints from Answer 3.16. It is not a screenshot from ERwin or Rational Rose.

UNIVERSITY: organization

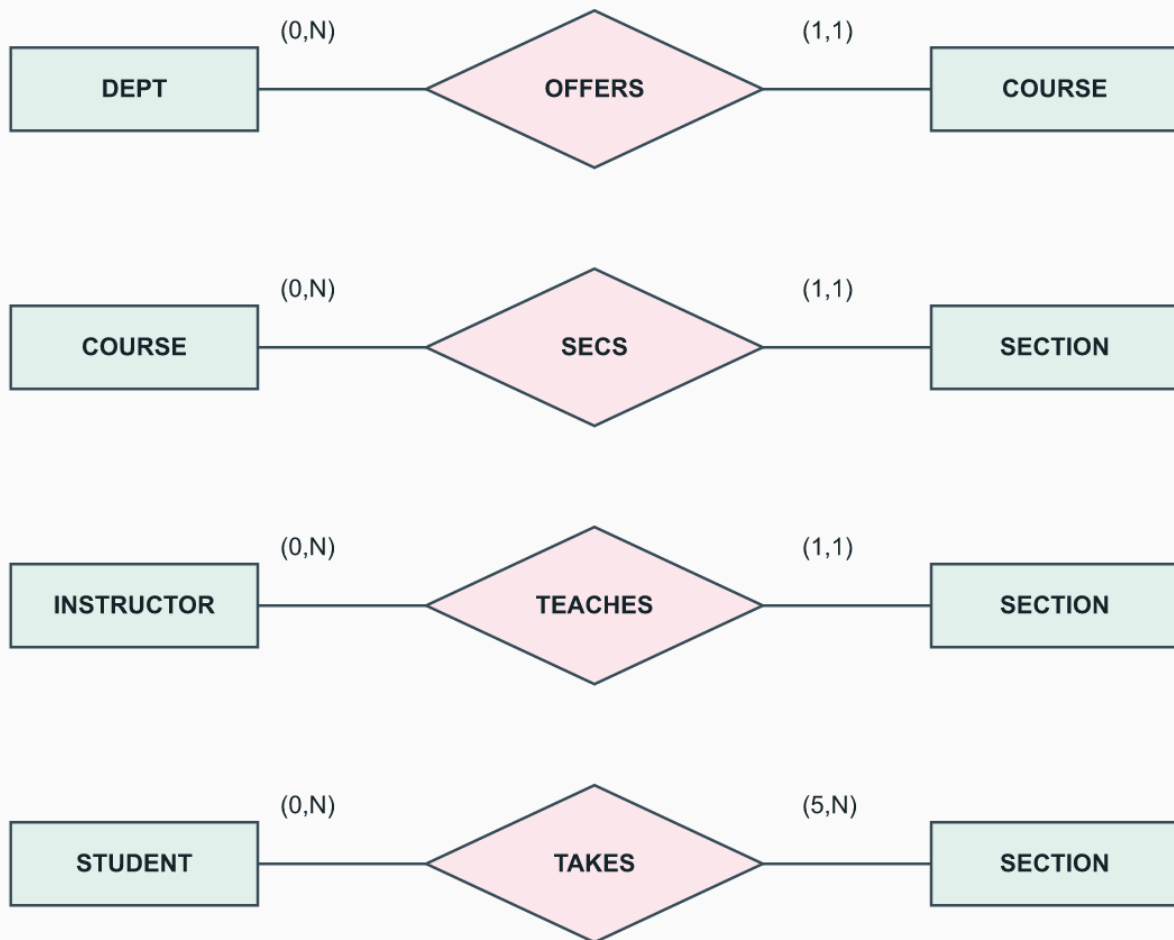
ER: counts beside each entity



HAS follows Figure 3.20; the prose requires one department. Both CCode and CoName are independent keys.

UNIVERSITY: teaching

ER: counts beside each entity



HAS follows Figure 3.20; the prose requires one department. Both CCode and CoName are independent keys.

Type	Independent keys	Other attributes
COLLEGE	CName	COffice, CPhone
DEPT	DCode; DName	DOffice, DPhone
COURSE	CCode; CoName	Level, Credits, CDesc
INSTRUCTOR	Id	IName, IOffice, IPhone, Rank
STUDENT	Sid	SName(FName,MName,LName), Addr, Phone, Major, DOB
SECTION	SecId	SecNo, Sem, Year, CRoom(Bldg,RoomNo), DaysTime

Grade belongs to TAKES and CStartDate to CHAIR. Both CCode and CoName are underlined in the photographed Figure 3.20. For HAS, the figure permits zero departments while the prose requires one; the diagram here follows the figure. Select the prose variant by changing only the student's minimum to one.

Validation case	Expected result
Section has five distinct students, one course, one teacher	Pass the local participation checks.
Remove one student	Fail minimum five.
Give a section a second instructor	Fail TEACHES.
Give a course an existing CoName but a different CCode	Fail independent CoName uniqueness.
Two distinct SecIds share course, term, and SecNo	Fail extra local-number uniqueness.
One student takes two sections at the same term/time	Fail Answer 3.16(b).

To reproduce in a modeling application: enter entity types and attribute/key definitions, add the nine relationships, choose one consistent notation, attach the two relationship attributes, and record constraints not expressible by local cardinalities as model rules. Export the model and check the cases above. Actual execution in the named proprietary tools remains unverified.

3.32. MAIL_ORDER Lab

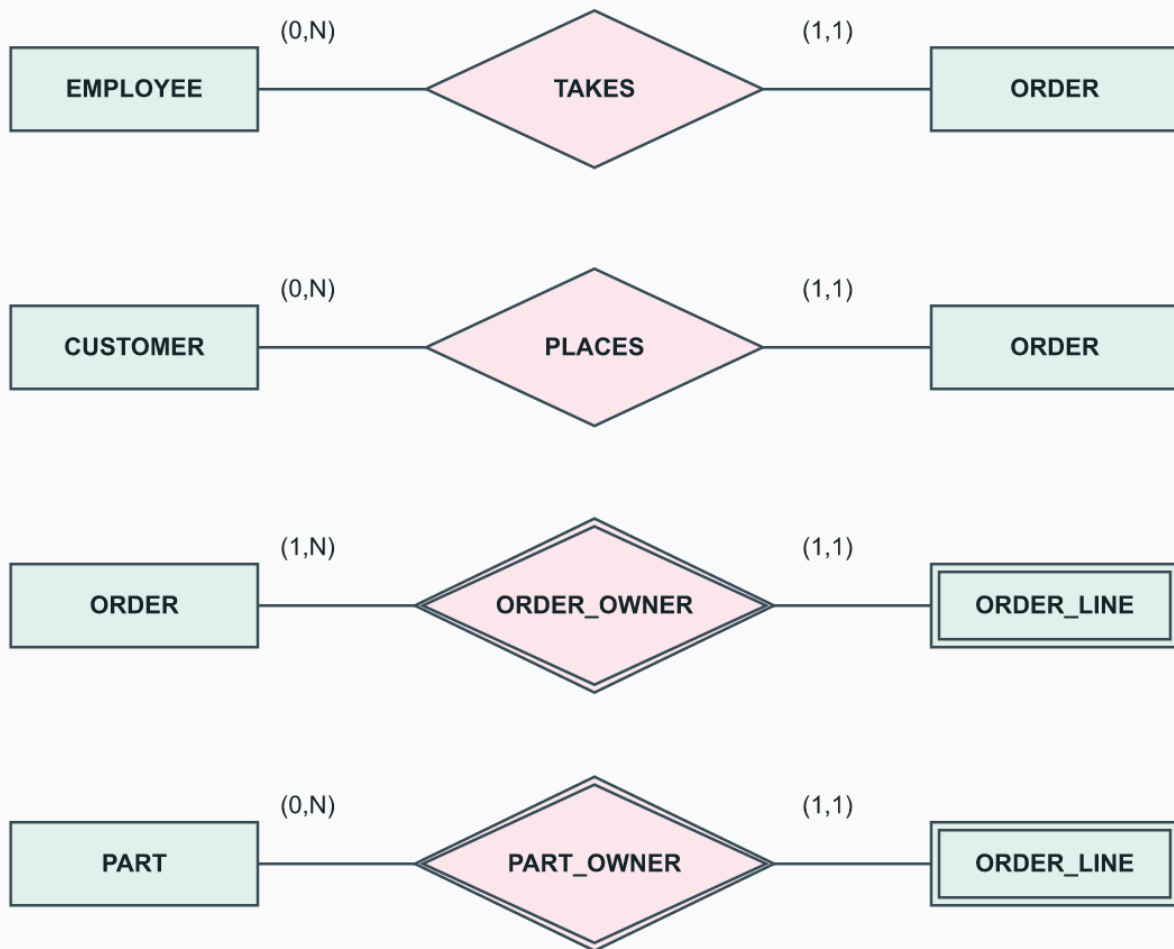
Source: laboratory exercise 3.32, p.133.

Completion: conceptual answer supplied; required modeling-application work not completed. No ERwin/Rational Rose model file or tool execution is supplied.

Type	Key/partial key	Attributes
EMPLOYEE	EmployeeNo	FirstName, LastName, ZipCode
CUSTOMER	CustomerNo	FirstName, LastName, ZipCode
PART	PartNo	PartName, Price, QuantityInStock
ORDER	OrderNo	ReceiptDate, ExpectedShipDate, ActualShipDate
ORDER_LINE (weak)	ORDER and PART jointly; no separate partial key	Quantity

Order lines have two identifying owners

ER: counts beside each entity



ORDER and PART jointly identify ORDER_LINE. Quantity belongs to that line; there is no independent partial key.

ORDER_LINE has two owners, ORDER and PART; one line exists per order-part pair. It has no independent partial key because that pair suffices. This is one conceptual alternative to an M:N CONTAINS relationship with Quantity. Each order is taken by one employee, placed by one customer, and has at least one line. Employees/customers/parts can exist without an order in this model.

Order	Part	Quantity
O1	P1	2
O1	P2	3
O2	P1	1

Quantity belongs to the order-part combination, not just PART. Assume positive integer quantities, nonnegative stock, and one currency per database. ActualShipDate can be absent until shipment. Historical sales price is NOT supplied by the prompt: add UnitPriceAtOrder only if historical billing is required. Do not silently assume current PART.Price is the price originally charged.

Model-tool steps are the same as Answer 3.31: create keys, relationships and Quantity, then test an order with zero lines (reject), a repeated order-part identity (reject), and the same part in two orders (allow). These are conceptual checks; no live inventory is changed.

3.33. MOVIE Lab with Roles and Quotations

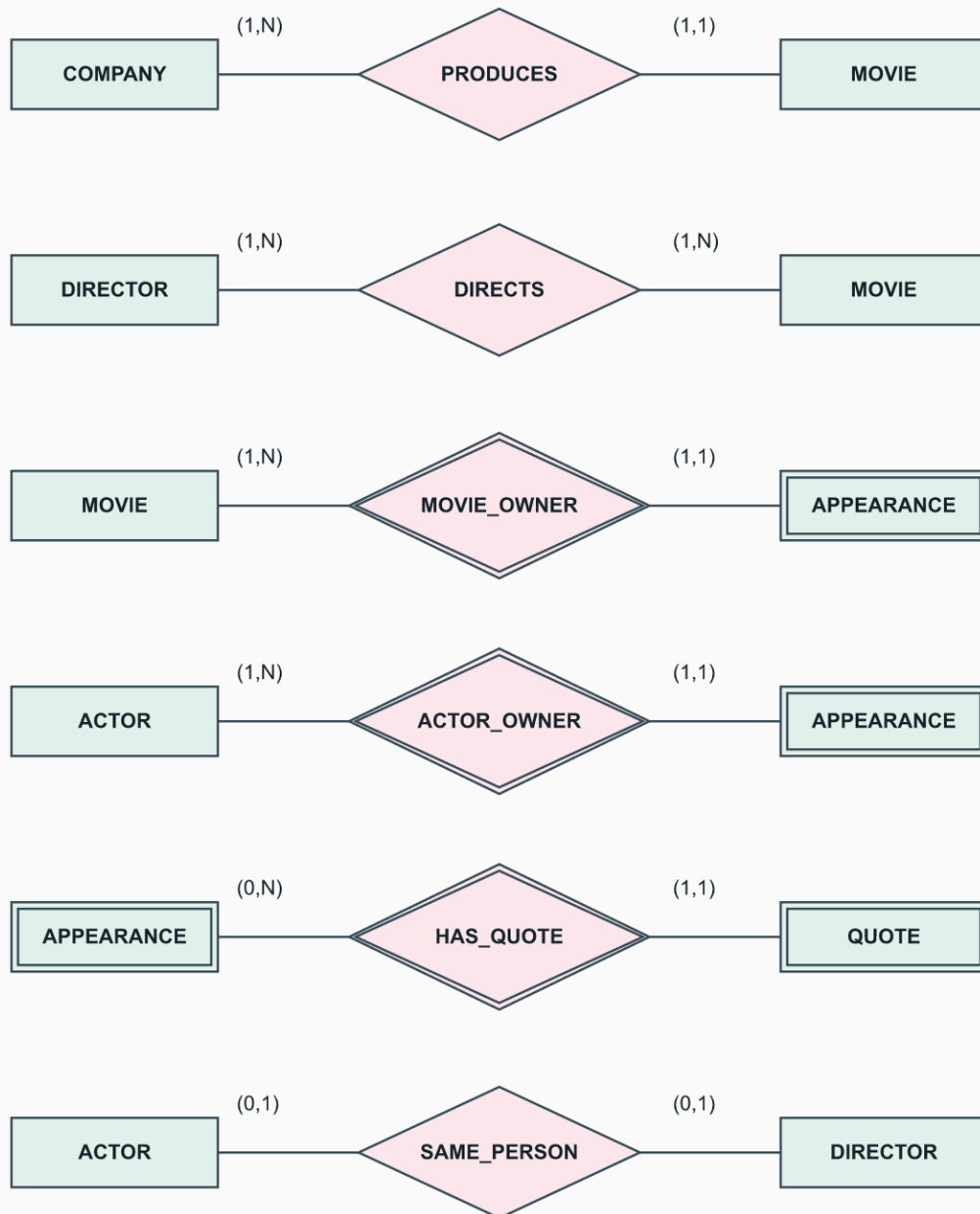
Source: laboratory exercise 3.33, pp.133-134.

Completion: conceptual answer supplied; required modeling-application work not completed. No ERwin/Rational Rose model file or tool execution is supplied.

Type	Identifier	Attributes
MOVIE	(Title,Year), composite	Length in minutes, PlotOutline, Genres (multivalued)
ACTOR	(Name,DateOfBirth), composite	As supplied by the exercise
DIRECTOR	(Name,DateOfBirth), composite	As supplied by the exercise
COMPANY	Name	Address
APPEARANCE (weak)	MOVIE and ACTOR together	Role
QUOTE (weak)	QuoteNo within APPEARANCE	QuoteText

Quotations belong to an actor's movie appearance

ER: counts beside each entity



MOVIE and ACTOR jointly identify APPEARANCE. QuoteNo is partial within an appearance.

Every movie has at least one actor, director, genre, and one production company. Every represented actor/director/company is connected to at least one movie, as the exercise requires. ACTOR and DIRECTOR may overlap; SAME_PERSON is an optional 1:1 correspondence, not disjointness.

The exercise uses singular "a role"; this design stores one Role per actor/movie pair. If an actor plays multiple roles, make Role multivalued or introduce ROLE_APPEARANCE with a local role discriminator and attach quotes there.

Movie	Actor	QuoteNo	Illustrative quote text
(Sample Film,2026)	(Alex Example,1990-01-01)	1	The door is open.
(Sample Film,2026)	(Alex Example,1990-01-01)	2	We should return.

These invented sentences are not quotations from a real film. A quote belongs to an actor's appearance in that movie, not to any actor independently of the movie. A movie may have zero quotes. Identical text spoken twice can have different QuoteNo values; text alone is not a reliable identifier. For a quote whose speaker did not appear in the film, reject the missing owner. Do not use the one-director maximum from Figure 3.25: this lab explicitly permits multiple directors.

3.34. CONFERENCE_REVIEW Lab

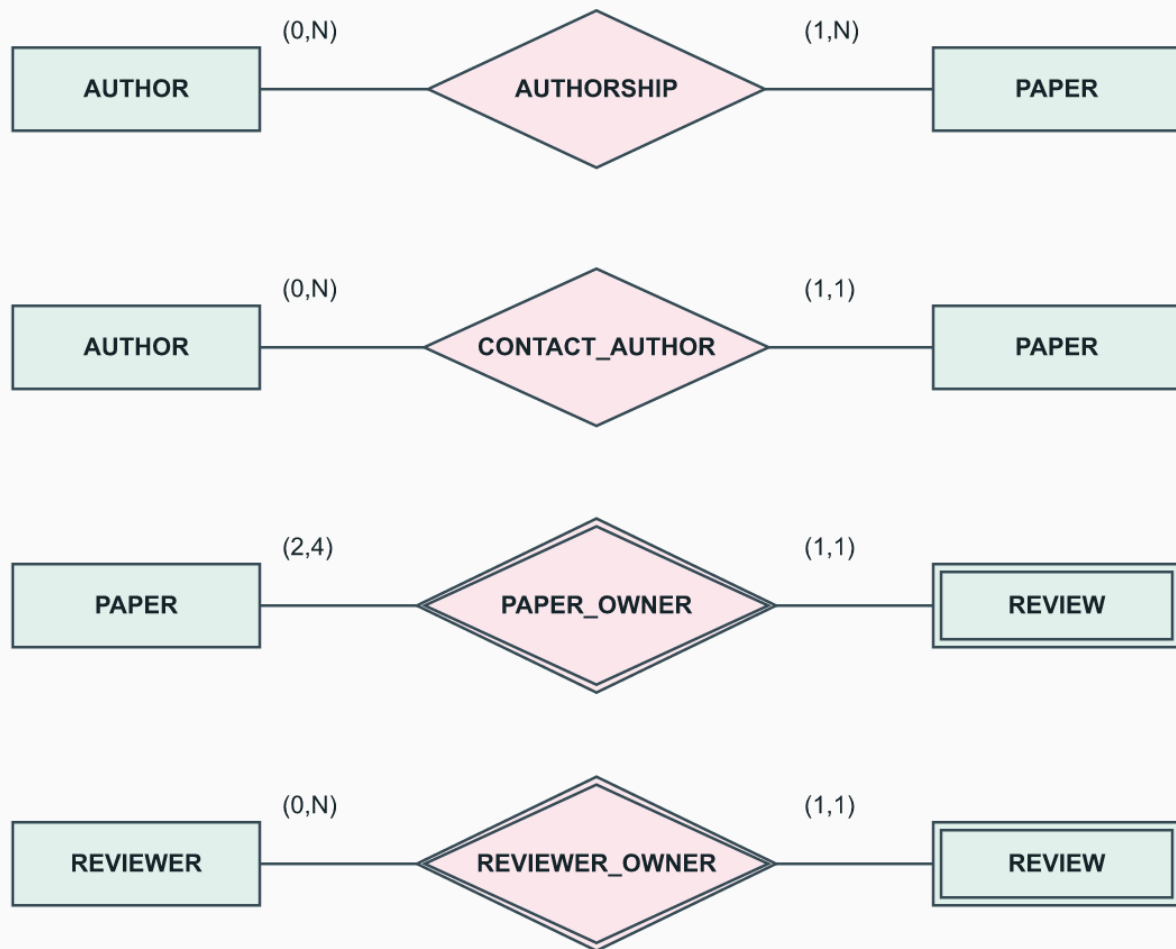
Source: laboratory exercise 3.34, p.134.

Completion: conceptual answer supplied; required modeling-application work not completed. No ERwin/Rational Rose model file or tool execution is supplied.

Type	Key	Attributes
AUTHOR	Email	FirstName, LastName
PAPER	PaperId	Title, Abstract, FileName
REVIEWER	Email	FirstName, LastName, Phone, Affiliation, Topics (multivalued)
REVIEW (weak)	PAPER and REVIEWER together	TechnicalMerit, Readability, Originality, Relevance, Recommendation, CommitteeComment, AuthorComment

Conference reviews retain their paper and reviewer

ER: counts beside each entity



PAPER and REVIEWER jointly identify REVIEW. Contact author must also be an author; access control is separate.

AUTHORSHIP is M:N, with at least one author per paper. CONTACT_AUTHOR gives each paper exactly one contact who must also occur in that paper's AUTHORSHIP. REVIEW is identified by the paper-reviewer pair; a paper has two to four assigned review records, and a reviewer can have none or many. Each review belongs to exactly one paper and one reviewer.

Scores are integers 1 through 10 when entered. Recommendation states whether the reviewer recommends acceptance or rejection. A simple {Accept,Reject} encoding is one design choice; the prompt does not prescribe a finer recommendation scale. For this answer's optional pending-review state, scores/recommendation/comments may be missing. A completed review must contain all four ratings, a recommendation, CommitteeComment, and AuthorComment. Allowing pending records is an explicit design extension, not permission to omit the requested comments from a completed review. Missing ratings are not zero.

Check	Outcome
One assigned reviewer	Reject required assignment count.
Two distinct reviewers	Meets the assignment minimum.
Five reviewers	Reject maximum four.
Duplicate (PaperId,ReviewerEmail)	Reject duplicate review identity.
Rating 11	Reject rating domain.
Contact author not in authorship	Reject subset constraint.
Mark complete without AuthorComment	Reject completion; the author-facing comment is missing.
All four scores, recommendation, and both comments present	Meets the stated completion requirements.

AUTHOR and REVIEWER can represent the same person; identical email can connect their identities. Do not assume disjoint roles. A prohibition on reviewing one's own paper is a sensible optional policy, but the exercise does not explicitly state it. CommitteeComment must be protected from the author-facing view; an ER diagram alone does not enforce access control. Use fabricated emails and file names in a teaching instance, not real submissions or reviewer data.

3.35. AIRLINE Lab

Source: laboratory exercise 3.35, p.134; Figure 3.21, p.128. The complete entity and relationship inventories are in Answer 3.19.

Completion: conceptual answer supplied; required modeling-application work not completed. No ERwin/Rational Rose model file or tool execution is supplied. Use all three diagrams in 3.19: owner hierarchy, scheduled/actual airports, and aircraft types/assignment. Together they cover all eleven displayed relationships; the identity illustration below alone is not the complete AIRLINE schema.

The same seat number can occur under different dated owners

Illustrative reservations: same flight and leg

Flight / leg	Date	Seat	Decision
F100 / 1	2026-09-10	12A	First reservation: allow
F100 / 1	2026-09-11	12A	Different owner: allow
F100 / 1	2026-09-10	12A	Same full identity: reject

Date is part of the owner identity. Seat 12A alone cannot identify a reservation.

Object	Illustrative complete identity
Flight	F100
Leg	(F100,1)
Dated instance	(F100,1,2026-09-10)
Seat reservation	(F100,1,2026-09-10,12A)
Fare	(F100,ECON)

Test	Result
Seat 12A on the same leg on the following day	Allowed: different dated owner.
A second 12A on the same leg/date	Reject duplicate identity.
Reservation without the dated leg instance	Reject missing owner.
An instance with no actual departure yet	Allowed by the figure's optional DEPARTS.
An instance without an assigned airplane	Reject ASSIGNED's total participation.

In the modeling application, create regular entities first, then the weak hierarchy, then the remaining airport/type/assignment relationships and their attributes. Keep scheduled and actual times separate. Enter every min-max rule listed in Answer 3.19 and verify the sample identities above. The locally generated diagrams and conceptual checks are reproducible; no claim is made that the model was entered in ERwin or Rational Rose. A live reservation system would additionally require temporal, capacity, payment, and transaction rules outside this exercise.

Coverage and Limits

Answers 3.1-3.15 address the review questions and 3.16-3.30 the written exercises. Each subpart of 3.16, 3.23, 3.27, 3.28, and 3.30 is addressed. Answers 3.31-3.35 provide conceptual solutions, inventories, and checks, but the five laboratories' modeling-application deliverables remain uncompleted. Do not report those five original laboratory tasks as fully completed merely because their diagrams exist. This document does not assign the excluded UML section to students; Answer 3.30 is included because the instructor requested all textbook exercises. Several schema choices need explicit assumptions. The stated alternatives are not uniquely determined publisher answers, and passing local tests does not prove every possible design correct.